清 华 大 学

综 合 论 文 训 练

题目：基于随机梯度蒙特卡洛采样的贝叶斯神经网络应用研究

系　　别：电子工程系

专　　业：电子信息科学与技术

姓　　名：张亦弛

指导教师：欧智坚　副研究员

2017 年 6 月 22 日

关于学位论文使用授权的说明

本人完全了解清华大学有关保留、使用学位论文的规定，即：学校有权保留学位论文的复印件，允许该论文被查阅和借阅；学校可以公布该论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存该论文。

(涉密的学位论文在解密后应遵守此规定)

签　名：_____导师签名：_____日　期：_____

中文摘要

随机梯度马尔可夫链蒙特卡洛采样方法（SG-MCMC）是近几年发展起来的一类能够在高维状态空间中进行有效采样的方法，通过对神经网络参数后验概率分布的良好估计，在贝叶斯神经网络的研究中取得了广泛应用。然而，由于存在模型存储代价大和推理过程中计算时间长这两方面问题，基于 SG-MCMC 的贝叶斯神经网络很难在实际问题中进行应用。本研究针对上述问题，提出了基于权重剪枝的网络压缩和模型参数重训练两个解决方案，解决了贝叶斯神经网络应用的难题，同时给出了一种结合 SG-MCMC 采样和随机优化的训练方法，以及综合 SG-MCMC 采样、网络剪枝和网络参数重训练的一种新的学习框架，并在基于 MNIST 数据集的手写体识别问题中进行了实验验证，在模型只有 1/16 大小的情况下得到了比现有方法相对降低 10%以上的测试错误率。

关键词：贝叶斯神经网络；随机梯度马尔可夫链蒙特卡洛采样；模型压缩；网络重训练

ABSTRACT

Stochastic Gradient Markov Chain Monte Carlo (SG-MCMC) is a kind of method which can do sampling effectively in high-dimensional state space developed in recent years. Account for the ability to do estimation of the posterior probability distribution of neural network parameters, SG-MCMC methods have been widely used in the study of Bayesian neural network. However, it is difficult to apply the Bayesian neural network based on SG-MCMC to practical problems due to the existence of high cost of model storage and long calculation time to do inference. In order to solve the above problems, this paper proposes two solutions based on weight pruning network compression and model parameter re-training, which solves the problem of Bayesian neural network application. At the same time, a new method which combines SG-MCMC sampling and stochastic optimization, and a new learning framework for SG-MCMC sampling, network pruning and network parameter re-training is proposed. Experimental results is based on the handwriting recognition task on MNIST data set. While doing inference in the new framework, the test error rate is reduced by more than 10% comparing to the existing method, with only 1/16 of model size.

Keywords: Bayesian Neural Network; SG-MCMC; network compression; network re-training

目 录

第 1 章 引 言 1
1.1 研究背景 1
1.1.1 贝叶斯神经网络概述 2
1.1.2 早期研究 4
1.1.3 研究现状 5
1.2 课题研究的意义和目标 6
1.2.1 研究贝叶斯神经网络应用的意义 6
1.2.2 研究工作介绍 6
1.3 论文结构 7
第 2 章 基本理论框架 9
2.1 基于蒙特卡洛采样的贝叶斯神经网络基本原理 9
2.1.1 贝叶斯神经网络框架 9
2.1.2 蒙特卡洛积分 10
2.2 蒙特卡洛采样 11
2.2.1 马尔可夫链蒙特卡洛采样的基本原理 11
2.2.2 经典马尔可夫链蒙特卡洛采样方法 12
2.2.3 随机梯度郎之万动力学 13
2.2.4 其他 SG-MCMC 采样方法 16
第 3 章 贝叶斯神经网络应用 19
3.1 参数先验概率分布的选择 19
3.2 应用模型 20
3.2.1 深度神经网络模型 20
3.2.2 长短时记忆网络模型 21
3.3 针对应用难点的解决方案 22
3.3.1 基于网络剪枝进行模型压缩的原理 22
3.3.2 网络参数重训练 23
第 4 章 实验结果 25

4.1 实验基本情况 25
4.1.1 实验环境 25
4.1.2 工程实现 25
4.2 验证性实验结果 27
4.2.1 对给定分布进行采样 27
4.2.2 曲线回归 29
4.2.3 基于 DNN 的 MNIST 手写体识别 32
4.2.4 基于 LSTM 的语言模型建模 34
4.3 创新性实验结果 35
4.3.1 基于网络剪枝的模型压缩 35
4.3.2 网络参数重训练 37
第 5 章 结论与展望 39
插图索引 41
表格索引 42
参考文献 43
致　　谢 46
声　　明 47
附录 A：几种 SG-MCMC 采样算法流程图 49
附录 B：外文资料的调研阅读报告或书面翻译 52

第1章　引 言

1.1　研究背景

在一个机器学习任务中，我们通常需要建立某种模型，给予这个模型足够的训练数据，使其学习到这些数据之间的关系和规律，并利用这些学到的“知识”对新的数据做出预测与推理。

近年来，随着深度学习的兴起，神经网络（Neural Network, NN）模型成为了机器学习中应用最多的模型，在诸如数据挖掘、计算机视觉、自然语言处理等领域的众多任务中，都取得了令人感到不可思议的成功。神经网络是一种经常被应用于解决计算机科学领域中各种工程问题的“非线性统计数据建模工具”。神经网络模型可以表达一种从输入到输出的映射，在选择适当的网络结构，给予足够的训练数据，用合适的训练方法进行模型优化后，理论上可以表达任意的映射，这也正是神经网络的强大之处和应用广泛的原因所在。

在经典的神经网络模型中，网络给出的映射是通过确定的网络参数唯一确定的，换言之，当我们给神经网络相同的输入时，得到的输出也一定相同。然而有些时候，模型对于自己的输出过于自信并不是一件好事。例如，当训练数据量较小，难以涵盖可能出现的所有情况时，即使新的测试数据恰好处在模型训练的“盲点”，我们的模型却仍然能够自信地做出预测，显然在现实中，这并不是一个很好的结果。

将神经网络贝叶斯（Bayesian）化，使模型拥有表达不确定度（Uncertainty）的能力，可以在很大程度上解决上述问题。在贝叶斯神经网络（Bayesian Neural Network, BNN）中，网络参数不再是确定的数，而是服从某种概率分布的随机变量，如图 1.1 中所示。此时，给定一个神经网络的输入，我们得到的就不再是一个确定的输出，而是给定输入条件下输出的一个条件分布，也就是对预测结果不确定度的表达。回到刚才的例子，如果模型在给出预测的同时，也会同时表达出这个预测的不确定度有多大，那对于处于训练“盲点”的测试数据，预测结果的不确定度也就会相应较大，这样的预测结果也就更加符合常理。在贝叶斯神经网络中，研究的就是如何利用这种不确定度，“让模型知道它们什么时候不知道”，从而更好地进行预测与推理。

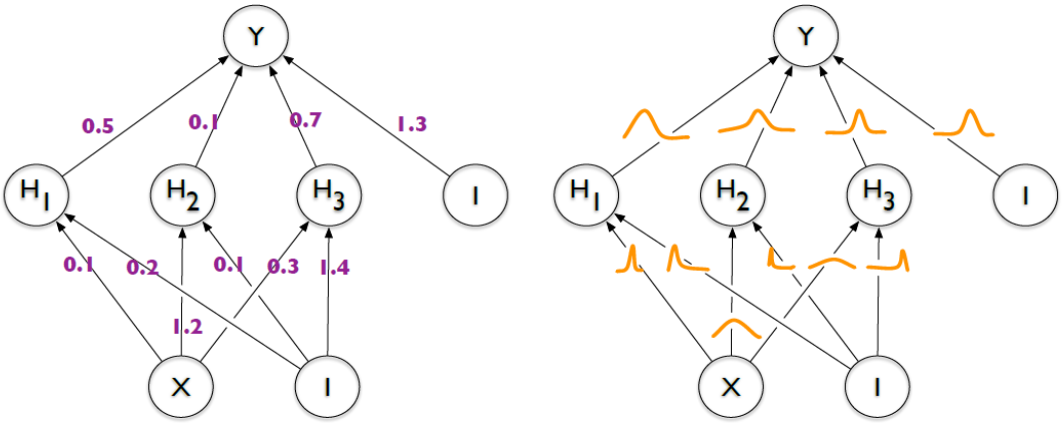

图 1.1 经典神经网络和贝叶斯神经网络的区别[①]

1.1.1 贝叶斯神经网络概述

贝叶斯是机器学习、统计模式识别和数据分析领域中的一种通用框架。假设在贝叶斯的观点中，我们对于模型参数的分布有一个假设，称为参数的先验概率分布（prior distribution）。在给出训练数据后，通过应用贝叶斯定理（公式 2-1），模型参数的分布会根据观测到的数据来进行调整，得到的新分布称为参数的后验概率分布（posterior distribution）。在进行推断时，需要使用所有满足后验概率分布的参数进行预测，再在整个参数空间中进行积分，计算出预测分布（prediction distribution），作为模型的预测结果。

贝叶斯神经网络就是将贝叶斯方法应用于传统的神经网络模型，通过推断神经网络模型参数的分布，为其提供一种概率上的表达方式[2]。在贝叶斯神经网络中，如何估计后验概率分布，并在推断时使用后验概率分布计算预测分布，是贝叶斯神经网络的核心问题。不同于一些简单模型，神经网络的参数处于一个很高维的空间中，其后验概率分布没有闭式解，高维空间下的积分也没有办法直接计算。贝叶斯网络在实际应用中，只能通过一些方法来近似估计网络参数的后验概率分布，使用这个近似的后验概率分布来进行预测与推理。

后验概率分布的近似估计方法是贝叶斯神经网络研究的焦点。事实上，贝叶斯神经网络的训练过程，就是在做网络参数的后验概率分布的近似估计。那么，如何估计一个在高维空间中的、没有闭式解的概率分布呢？

蒙特卡洛方法给出了一种解决方案。蒙特卡洛方法是一系列随机模拟方法的

---

① 图片引自参考文献[1]

总称，我们所用到的方法，更准确来讲叫做马尔可夫链蒙特卡洛（Markov Chain Monte Carlo, MCMC）方法。这种方法的基本思想是构造一条平稳分布满足我们想要估计的目标分布的马尔可夫链，利用马尔可夫链的状态转移在状参数空间中进行采样，所采到的样本的分布就是目标分布的近似估计结果。而随机梯度马尔可夫链蒙特卡洛（Stochastic Gradient Markov Chain Monte Carlo, SG-MCMC）采样方法——简称为随机梯度蒙特卡洛采样，则是在马尔可夫链蒙特卡洛方法的基础之上，在状态转移时增加了梯度信息的利用，可以有效地在高维神经网络参数空间中进行采样，并通过结合随机近似得以应用于大型的训练数据集。由于上述两个特点，基于随机梯度蒙特卡洛采样的贝叶斯神经网络模型能在模型参数很多、复杂度高和使用大量数据训练的情况下很好地完成网络的训练，从而应用于各种深度学习任务。

变分推断（Variational Inference, VI）是另一种被研究最多的后验概率分布近似估计方法。变分推断的思想是使用一个简单的、有显示表达的分布来近似目标分布，称为变分分布，目标是最小化变分分布和目标分布之间的 Kullback-Leibler（KL）距离[①]，从而将后验概率的估计问题转化为优化 KL 距离目标函数的网络参数优化问题。不同于经典神经网络优化所进行的参数的点估计，变分推断是对参数的分布进行优化，使得贝叶斯神经网络的不确定度得以保留。

马尔可夫链蒙特卡洛和变分推断是目前可以在贝叶斯神经网络中进行实际应用的两种主流方法。两种方法各有其优缺点：马尔可夫链蒙特卡洛方法的优势在于其对后验概率的分布更加准确，当采样的样本数量增加到足够多时，样本的分布可以收敛到真实的后验概率分布，而变分推断受限于变分分布的形式，如果变分分布的空间中并不包含真实的后验概率分布，就算参数优化到最优也会存在估计偏差。此外，马尔可夫链蒙特卡洛方法的实现更简单，在现有经典神经网络的各种训练方法上稍加修改即可实现采样，具体参见 2.2 节。马尔可夫链蒙特卡洛的不足之处，在于其需要采样后再对模型参数的样本做平均，使用模型进行推断需要的计算时间和模型存储需要的空间都成倍增加，变分推断方法则没有这么高的成本，这是变分推断的优势所在。

接下来的两节，我会从贝叶斯神经网络以及上述两种后验概率分布近似估计方法研究的历史和现状，对前人的工作进行梳理与总结。

---

① Kullback-Leibler 距离又称为相对熵，是两个概率分布之间差异情况的一种度量准则。

1.1.2 早期研究

早在上世纪 80 年代末期，就已经有了贝叶斯方法和神经网络结合的雏形[3][4]，其中 Tishby 等人的论文[3]中提出了为网络参数做先验概率分布假设的想法，一般被认为是最早的贝叶斯神经网络。1991 年，Denker 和 LeCun 在[3]的基础上，提出了用拉普拉斯近似（Laplace Approximation）的方法进行贝叶斯神经网络参数后验概率分布的估计，同时他们也将如今几乎所有神经网络的训练都会采用、而当时刚刚被提出的反向传播算法应用在了贝叶斯神经网络中[5]。然而，拉普拉斯方法对于后验概率分布估计的近似误差较大，效果并不理想，之后也没有得到太多发展。

1992 年，MacKay 在他的论文[6]中完善了贝叶斯神经网络的理论基础，并给出了其应用的一个实用框架，标志着贝叶斯神经网络的研究进入一个快速发展的黄金时期。1993 年，Neal 在他的论文[7]中将 Duane 等人 1987 年提出的混合（哈密尔顿）蒙特卡洛（Hybrid/Hamiltonian Monte Carlo, HMC）方法[①]应用到了贝叶斯神经网络中，迈出了将马尔可夫链蒙特卡洛方法应用于贝叶斯神经网络的第一步。哈密尔顿蒙特卡洛方法将参数视为一个哈密尔顿动力学系统中的粒子，由粒子在动力学系统中势能的梯度决定粒子的运动方向，对应参数在参数空间中的转移，能够在高维空间中进行有效采样。该领域之后几乎所有的方法也都沿用了基于梯度的思想，因此也有人称之为基于马尔可夫链蒙特卡洛采样的贝叶斯神经网络的“黄金标准”方法。2 年后，Neal 在他的博士论文[8]中对贝叶斯神经网络和马尔可夫链蒙特卡洛方法的应用做了详细的阐述和总结，为基于马尔可夫链蒙特卡洛方法的贝叶斯神经网络提供了一套理论基础和应用框架，也让这种方法真正具有了实用的价值。

也是在 1993 年，变分推断被 Hinton 和 Van Camp 应用到了贝叶斯神经网络中[10]，他们使用的对角高斯变分近似法由 Barber 和 Bishop 在 1998 年发表的论文[11]中扩展到了全协方差高斯变分近似，成为变分推断实际应用于贝叶斯神经网络的开端。

---

① Neal 最开始沿用了 Hybrid Monte Carlo 这个名字，但由于这种方法的核心是哈密尔顿动力学，Hamiltonian Monte Carlo 能够更加准确、直观地反映这种方法的特点，Neal 在自己之后的著作[9]中将其更名为 Hamiltonian Monte Carlo，沿用至今。

1.1.3 研究现状

论文[12][1][13]是基于变分推断方法的最近研究工作，对传统的变分推断方法做了程度的改进，并在更复杂的模型和更复杂的任务中进行了应用。另一种方法期望传播（Expectation Propagation, EP）是变分推断方法的一个变种，优化的是 KL 距离的对称形式（KL 距离本身不具有对称性），具有代表性的工作是 2015 年 Hernandez-Lobato 发表的论文[14]。Yarin 在他 2016 年底发表的博士论文[2]中对以上这些工作进行了总结，并对变分推断方法的最新研究成果进行了详尽系统的整理与归纳。由于变分推断方法并不是本论文研究的重点，这里就不展开做详细的讨论了。

结合随机梯度是马尔可夫链蒙特卡洛（以下简称 SG-MCMC）方法近期的研究方向。2011 年，Welling 和 Teh 提出了第一种 SG-MCMC 采样方法——随机梯度郎之万动力学（Stochastic Gradient Langevin Dynamics, SGLD）[15]，正式开启了基于随机梯度蒙特卡洛采样应用于贝叶斯神经网络的研究。郎之万动力学也是物理学中对分子系统中分子运动的一种建模方式，是哈密尔顿动力学不考虑粒子动量时的简单情况。这种方法的创新之处在于，不再像哈密尔顿蒙特卡洛采样那样使用整个数据集计算梯度，而是在每次迭代时只使用训练集的一个小子集去计算梯度，根据这个随机梯度进行参数的状态转移。SGLD 采样方法的也实现极为简单，就是在神经网络训练最为常见的随机梯度下降法上对求出的梯度加一个随机噪声，即可完成对于参数空间的探寻，并由作者证明随着学习速率的衰减，会收敛于真实的后验概率分布。因为 SGLD 具有简单、需要调整的参数少、稳定性高等优势，目前仍然是基于 SG-MCMC 采样的贝叶斯神经网络研究中的主流方法之一，也是该领域其他很多工作的基准。

SGLD 之后，SG-MCMC 采样方法的研究开始快速发展，Chen 等人在 2014 年将 HMC 方法结合随机梯度，提出了 SGHMC 方法[16]。杜克大学一个从事该领域的研究组，又在 SGHMC 的基础上，通过增加控温器——一个控制动量的全局参数，提出了 SGNHT 方法[17]；在 SGLD 的基础上增加一种自适应的预处理条件（precondition），得到 pSGLD 方法[18]；以及将采样和随机优化方法相结合的 SANTA 方法[19]等。这些方法的原理以及和随机优化方法的关系将会在 2.2 节中进行介绍与分析。

在 SG-MCMC 采样方法快速发展的同时，基于 SG-MCMC 的贝叶斯神经网络在图像识别、自然语言处理等领域实际问题中的应用研究，也在如火如荼地进行，并取得了很不错的结果[20] [21]。虽然贝叶斯方法的应用对于解决很多实际问

题都可以带来结果上的提升，但由于该方法的应用离不开采样和在推断时对各个模型预测结果的平均，计算量大和存储空间需求高的问题在很大程度上制约着其应用。在接下来的一节里会对这个问题进行分析，并给出本研究的意义和期望解决的问题。

1.2 课题研究的意义和目标

1.2.1 研究贝叶斯神经网络应用的意义

在基于采样的贝叶斯神经网络中进行推断时，每一个网络参数样本能提供一个预测结果，这些预测结果的统计量可以作为贝叶斯神经网络中最为重要的不确定度的度量。然而，如何评估不确定度的好坏呢？在目标单一的机器学习问题中，可能很难利用贝叶斯神经网络提供的不确定度，需要将对网络的预测结果应用到下游的其他任务中，不确定度好坏的价值才能得以体现。结合下游任务会使实验变得过于复杂庞大，不利于 SG-MCMC 采样方法本身的研究，因此在本研究中并不对于不确定度的好坏进行深入研究。

随着深度学习的发展，使用的模型变得越来越复杂。在使用有限的数据调整规模庞大的复杂模型时，很容易出现过拟合（overfitting）[①]的现象。过拟合现象可以看作模型对于自己的预测过于自信的结果，而在基于 SG-MCMC 采样的贝叶斯神经网络中，需要对于每一个采样得到的网络参数样本都做一次预测后，再对预测结果进行平均得到最终的预测，这种平均可以有效抑制过拟合的发生。从这个意义上来讲，贝叶斯方法的应用除了能带来网络预测结果不确定度的度量，也能在各种深度学习任务中取得更好的表现[②]。当预测不确定度的好坏难以评估时，性能上的提升成为本研究关注的焦点。

1.2.2 研究工作介绍

基于 SG-MCMC 采样的贝叶斯神经网络作为最近几年才发展出来的新领域，缺乏对于其中各种方法的系统梳理、总结和实验验证。目前也有一些对于各种 SG-MCMC 采样方法进行总结的文章[22][23]，但他们的关注点更多在于采样方法本

---

① 过拟合指模型在训练数据集上的表现很好，但对于没有见过的新测试数据却难以做出正确预测的现象。模型过于复杂而训练数据太少、训练集数据分布和测试数据不一致、过度训练等都可能导致模型过拟合。

② 例如分类任务中更高的预测准确率，回归任务中更小的均方误差等。

身而非 SG-MCMC 在贝叶斯神经网络中的应用，而且总结中没有包括 SG-MCMC 方法研究的最新进展。

在本研究中，理论部分会从几种 SG-MCMC 方法的原理出发，阐述各方法之间的区别和联系。实验部分首先通过一个对给定分布进行采样的仿真实验，对几种马尔可夫蒙特卡洛采样方法进行性能对比，然后对于基于 SG-MCMC 采样的贝叶斯神经网络应用于曲线回归、手写体识别和语言模型建模这三个实际的机器学习问题进行了研究。

在现有的基于 SG-MCMC 采样的贝叶斯神经网络框架中，为了更好地拟合后验分布概率，需要采尽可能多的网络参数样本。然而，在进行推断时，使用几个模型参数样本的预测结果进行平均，就意味着模型的存储空间和预测时需要的计算时间增大相应的倍数，难以在实际问题中进行应用。本研究主要致力于解决上述问题，做出了以下几点贡献：

1. 提出贝叶斯神经网络剪枝方法，解决模型存储的问题。
2. 提出网络参数重训练方法，解决推断时计算时间开销大的问题。这种方法脱离了基于采样的贝叶斯神经网络必须做模型平均的限制，为神经网络训练时在参数空间中的寻优提供了一种新思路。
3. 在基于 SG-MCMC 采样、网络剪枝和网络参数重训练的新框架下，针对 MNIST 手写体识别问题，在模型压缩超过 16 倍的同时，得到了更低的识别错误率。

1.3 论文结构

论文的第一部分是理论研究部分，包括第 2 章和第 3 章。第 2 章介绍基于 SG-MCMC 采样的贝叶斯神经网络的基本理论，首先在 2.1 节中介绍了贝叶斯神经网络的基本框架和使用蒙特卡洛积分进行贝叶斯推断的理论基础。2.2 节又分为四个小节，前两小节分别介绍了 MCMC 采样的基本原理和经典方法，然后在 2.2.3 节中阐述了 SG-MCMC 采样方法的原理和最简单的一种实现方法，最后在 2.2.4 节通过对几种 SG-MCMC 采样方法和随机优化方法的异同点的对比分析，总结了 SG-MCMC 采样方法和随机优化方法的关系。第 3 章主要围绕贝叶斯神经网络的应用展开。首先在 3.1 节中介绍了先验概率的形式和含义，然后 3.2 节中简要介绍了本论文研究的深度神经网络和长短时记忆网络两类模型。3.3 节又分为 2 小节，分别针对贝叶斯神经网络在应用时遇到的两方面困难，介绍了网络剪枝和网

络参数重训练这两种解决方案的思想与原理。

第二部分是实验结果部分，包括第 4 章。在 4.1 节，对采用的实验环境和各种理论方法的工程实现方式进行了概述。4.2 节给出了验证性实验的结果，包括四小节内容：4.2.1 和 4.2.2 节通过对给定分布进行采样和使用贝叶斯神经网络进行曲线回归这两个仿真实验，对几种 SG-MCMC 方法的采样性能进行了验证和对比分析。4.2.3 和 4.2.4 节分别针对基于 DNN 模型的 MNIST 手写体识别任务和基于 LSTM 的语言模型建模任务这两个实际问题进行研究，考察基于 SG-MCMC 采样的贝叶斯神经网络在这些实际问题中的表现。在 4.3 节中，基于手写体识别问题对贝叶斯神经网络的应用进行了更加深入的研究，给出了基于网络剪枝的模型压缩和网络参数重训练两方面工作的实验结果。

最后一部分是第 5 章，进行研究成果的总结和对该领域未来研究发展方向的展望。

第2章　基本理论框架

2.1　基于蒙特卡洛采样的贝叶斯神经网络基本原理

2.1.1　贝叶斯神经网络框架

在一个机器学习问题中，假设我们的训练集中有n个训练数据，输入记为$\mathbf{X} \triangleq \{\boldsymbol{x}_1, \boldsymbol{x}_2, \dots, \boldsymbol{x}_n\}$，对应的输出为$\mathbf{Y} \triangleq \{y_1, y_2, \dots, y_n\}$，则整个训练数据集记为$\mathcal{D} \triangleq \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \dots, (\boldsymbol{x}_n, y_n)$. 对于一个模型$f$，给定输入$\boldsymbol{x}$后输出$y = f_\theta(\boldsymbol{x})$，我们的目标是找到合适的模型参数$\theta$，使得在输入数据时尽可能产生相应的输出。

在贝叶斯框架中，模型参数有一个固有的概率分布$P(\theta)$，称为参数的先验分布（prior distribution）。通过贝叶斯定理：

$$P(\theta|\mathcal{D}) = \frac{P(\mathcal{D}|\theta)P(\theta)}{P(\mathcal{D})} \tag{2-1}$$

先验分布可以转化为后验概率分布（posterior distribution）$P(\theta|\mathcal{D})$. 其中，$P(\mathcal{D}|\theta)$称为似然函数，用来表征模型参数的似然性，也就是某一组模型参数有多大可能性产生训练数据中的输入输出。由于训练数据已经给定，$P(\mathcal{D})$仅仅起到归一化常数的作用，在实际训练中我们更习惯使用$P(\theta|\mathcal{D}) \propto P(\mathcal{D}|\theta)P(\theta)$来表达后验概率分布似然函数、先验概率分布之间的关系。从这个表达式中可以看出，后验概率分布综合了模型参数的先验知识和训练数据所提供的知识这两方面信息，寻找符合后验概率分布的模型参数θ，是我们希望寻找的满足“给定某一输入时更有可能产生期望的输出”的模型参数。

具体到神经网络模型中，后验概率分布的计算公式变为：

$$P(\theta|\mathcal{D}) = \frac{P(\mathbf{Y}|\mathbf{X}, \theta)P(\theta)}{P(\mathbf{Y}|\mathbf{X})} \tag{2-2}$$

这是由于在神经网络中，给定输入$\boldsymbol{x}$时，模型的输出$y = f_\theta(\boldsymbol{x})$可以解释为条件概率$P(y|\boldsymbol{x}, \theta)$，是神经网络进行概率建模的基础[24]。

似然函数$P(\mathbf{Y}|\mathbf{X}, \theta)$的计算可分为分类问题和回归问题两种情况。在分类问题中，网络输出y是一个维数等于类别数目的矢量，属于某一类别k的概率是由下式计算的 softmax 回归概率：

$$P(y=k|\boldsymbol{x},\theta)=\frac{\exp\left(f_\theta^k(\boldsymbol{x})\right)}{\sum_{\mathrm{d}'}\exp\left(f_\theta^{k'}(\boldsymbol{x})\right)} \tag{2-3}$$

其中$f_\theta^d(\boldsymbol{x})$表示网络在第$d$维上的输出值，$f_\theta^{d'}(\boldsymbol{x})$表示除第$d$维之外的输出值。在回归问题中，网络输出$y$是一个实数值向量，条件概率$P(y|\boldsymbol{x},\theta)$可以定义为一个均值为$f_\theta(\boldsymbol{x})$，方差为$\sigma_n^{-1}$的高斯分布：

$$P(y|\boldsymbol{x},\theta)=\mathcal{N}(y;f_\theta(\boldsymbol{x}),\sigma_n^{-1}\mathbf{I}\ ) \tag{2-4}$$

其中σ_n称为噪声精度，**I**表示单位阵。

在进行推断时，需要计算给定训练数据$\mathcal{D}$和新的输入$\boldsymbol{x}^*$条件下输出y^*的条件概率$P(y^*|\boldsymbol{x}^*,\mathcal{D})$，作为神经网络的预测分布。贝叶斯神经网络中，通过输出的条件分布在整个满足后验概率分布的模型参数空间中积分来计算预测分布：

$$P(y^*|\boldsymbol{x}^*,\mathcal{D})=\int P(y^*|\boldsymbol{x}^*,\theta)\,P(\theta|\mathcal{D})\mathrm{d}\theta \tag{2-5}$$

这种使用积分将模型参数边缘化的方法是贝叶斯神经网络的核心思想。在高维的参数空间中，2-5 式的积分没有闭式解，是无法直接进行计算的，下一节会介绍如何使用蒙特卡洛积分对其进行近似估计。

2.1.2 蒙特卡洛积分

经典神经网络的训练，是通过优化方法对网络参数进行寻优，即求解下面的优化问题：

$$\theta^{MAP}=\arg\max_\theta\sum_{i=1}^{N}\log P(y_i|\boldsymbol{x}_i,\theta)+\log P(\theta) \tag{2-6}$$

这种方法又称为最大后验概率法（Maximum a Posteriori, MAP），使用后验概率最大的参数作为神经网络参数的点估计。而在贝叶斯神经网络中，仅仅得到θ^{MAP}这样一个点是不够了，我们需要知道$P(\theta|\mathcal{D})$的分布，从而计算符合后验概率分布的参数在参数空间中的积分。在积分无法显示求解的情况下，蒙特卡洛积分（Monte Carlo Integration）就是近似求解 2-5 式积分的一种解决方案。

蒙特卡洛积分是在求没有闭式解的期望、积分的一种常用方法。其基本思想是使用从后验概率分布$P(\theta|\mathcal{D})$中采样有限个样本，使用每一个样本计算出的

$P(y^*|\boldsymbol{x}^*,\theta_i)$的平均值作为预测分布$P(y^*|\boldsymbol{x}^*,\mathcal{D})$的近似估计。设$\theta_1,\theta_2,\dots,\theta_n$满足$\theta_i \sim P(\theta|\mathcal{D})$，则有：

$$P(y^*|\boldsymbol{x}^*,\mathcal{D}) \approx \frac{1}{n}\sum_{i=1}^{n} P(y^*|\boldsymbol{x}^*,\theta_i) \tag{2-7}$$

大数定律保证了随着样本数的增加，采样平均的结果几乎必然收敛于真实的积分值，而通过中心极限定理则可以证明这种蒙特卡洛方法的估计误差正比于$1/\sqrt{\mathrm{m}}$，保证了使用蒙特卡洛方法进行积分近似估计的准确性。

在基于 SG-MCMC 采样的贝叶斯神经网络中，通过 SG-MCMC 方法在采样得到符合后验概率分布$P(\theta|\mathcal{D})$的样本，然后通过 2-7 式计算神经网络的预测分布，进行推断。

2.2 蒙特卡洛采样

2.2.1 马尔可夫链蒙特卡洛采样的基本原理

满足条件独立性（2-8 式），即每一个状态只依赖于前一个状态而和之前的所有状态无关的一系列随机变量$\theta_1,\theta_2,\dots,\theta_T$，称为一个马尔可夫链（Markov Chain）。

$$P(\theta_{t+1}|\theta_{\mathrm{t}},\theta_{\mathrm{t}-1},\dots,\theta_1) = P(\theta_{t+1}|\theta_{\mathrm{t}}) \tag{2-8}$$

概率$P(\theta_{t+1}=i|\theta_t=j)$称为马尔可夫链从状态$i$到状态$j$的转移概率。如果对于任意时刻$t$，转移概率$P(\theta_{t+1}=i|\theta_t=j)$都保持不变，则称该马尔可夫链具有齐次性。在一个齐次马尔可夫链中，满足 2-9 式的分布$\pi(\theta)$，称为该马尔可夫链的一个不变分布。

$$\pi(\theta') = \sum_{\theta} P(\theta'|\theta)\pi(\theta) \tag{2-9}$$

马尔可夫链具有如下的遍历性定理：对于一个非周期、任意两个状态均连通的马尔可夫链，给定任意的初始分布$\pi_0(\theta)$，经过足够长时间的转移后，都能最终收敛于唯一的不变分布$\pi(\theta)$，称为该马氏链的平稳分布。

这个定理是所有马尔可夫链蒙特卡洛（Markov Chain Monte Carlo, MCMC）采样方法的基础。MCMC 采样的目标是构造一个平稳分布和我们需要估计的目标分布，也就是贝叶斯神经网络的后验概率分布相同的马尔可夫链，在收敛后，通

过该马尔可夫链的状态转移即可得到符合后验概率分布的样本。

$$P(\theta'|\theta)\pi^*(\theta) = P(\theta|\theta')\pi^*(\theta'), \qquad \forall\theta,\theta' \tag{2-10}$$

那么，应该如何构造一个目标分布即为平稳分布的马尔可夫链呢？当马尔可夫链的状态转移概率满足 2-10 式时，很容易证明$\pi^*(\theta)$就是该马尔可夫链的平稳分布，即$\pi^*(\theta) = \pi(\theta)$。2-10 式称为马尔可夫链的细致平衡条件，其意义是在分布从状态θ转移到状态θ'而损失的概率质量，等于从状态θ'转移回状态θ得到的概率质量补充，说明分布$\pi^*(\theta)$是“平稳”的。因此，我们只要将马尔可夫链的转移概率构造为满足细致平稳条件，就可以达到目的。

根据马尔可夫链蒙特卡洛的原理，不难看出采样得到的样本并非相互独立的，而是和前一个状态有关。由于在实际应用中只能使用有限个样本估计后验概率分布，使用相关度高的样本会使估计结果产生偏差，应该选择相关性尽可能低的样本。如何选择样本，会在 4.2.2 节中进行讨论。

2.2.2 经典马尔可夫链蒙特卡洛采样方法

Metropolis-Hastings 算法（以下简称 MH 算法）给出了构造转移概率满足细致平衡条件的一种方法， 是最基本的一种 MCMC 方法，算法流程见表 2.1。

MH 算法的基本思想是通过对转移概率引入一个更正项$\alpha(\theta'|\theta)$，通过细致平衡条件对样本分布进行修正。很容易证明这样得到的分布$p(\theta)$满足平稳分布的条件：

$$\begin{aligned} p(\theta)q(\theta'|\theta)\alpha(\theta'|\theta) &= \min\big(p(\theta)q(\theta'|\theta), p(\theta')q(\theta|\theta')\big) \\ &= \min\big(p(\theta')q(\theta|\theta'), p(\theta)q(\theta'|\theta)\big) \\ &= p(\theta')q(\theta|\theta')\alpha(\theta|\theta') \end{aligned} \tag{2-11}$$

对于提议分布（proposal distribution）的选择，在 MH 算法中一般选择均值为当前状态θ_t，方差为定值的高斯分布，根据实际问题调整高斯分布方差的大小。由于新状态的提议是随机的，这种方法被形象地称为随机游走法。然而，当θ处在很高维的空间时，经常需要产生很多个提议状态，才可能发生一次状态转移，样本接受率低，采样效率低。因此，MH 算法——传统的 MCMC 采样方法难以在需要对高维参数空间进行采样贝叶斯神经网络中进行应用。

表 2.1　Metropolis-Hastings 算法流程[①]

算法 1　Metropolis-Hastings Algorithm

Input: proposal distribution $q(\theta'|\theta)$

Draw initial state θ_0 from $p(\theta_0)$

for $t = 0,1,\dots,T$, **do**

Draw a proposal state θ' from proposal distribution $q(\theta_{t+1}|\theta_t)$

Compute accept-reject probability $\alpha(\theta'|\theta_t) = \min\left(1, \frac{q(\theta_t|\theta')p(\theta')}{q(\theta'|\theta_t)p(\theta_t)}\right)$

Draw u from a uniform distribution of a range [0,1]

if $\alpha_t > u$ **then**

Accept proposal state: $\theta_{t+1} \leftarrow \theta'$

else

Reject proposal state: $\theta_{t+1} \leftarrow \theta_t$

end if

end for

2.2.3 随机梯度郎之万动力学

SG-MCMC 方法的目标是根据神经网络参数的后验概率分布$P(\theta|\mathcal{D})$在参数空间中进行有效采样。在神经网络训练时，通常将后验概率分布表示为如下的规范化形式：

$$P(\theta|\mathcal{D}) = \frac{1}{Z}\exp(-U(\theta)) \tag{2-12}$$

其中Z是归一化常数，$U(\theta)$是参数后验概率的负对数，称为势函数或势能。结合 2-2 式，有：

$$\begin{aligned} U(\theta) &= -\log P(\mathbf{Y}|\mathbf{X},\theta) - \log P(\theta) \\ &= -\sum_{i=1}^{N} \log P(y_i|\boldsymbol{x}_i,\theta) - \log P(\theta) \end{aligned} \tag{2-13}$$

$U(\theta)$的梯度可以表示为：

① 引用自参考文献[22]

$$f(\theta) \triangleq -\nabla U(\theta) = \sum_{i=1}^{N} \nabla_\theta \log P(y_i|\boldsymbol{x}_i, \theta) + \nabla_\theta \log P(\theta) \tag{2-14}$$

随机差分式（Stochastic Differential Equation, SDE）2-15 式称为郎之万等式（Langevin Equation），参数位置和势函数梯度满足该等式关系的系统称之为郎之万动力学系统（Langevin Dynamics）。

$$\mathrm{d}\theta = \frac{1}{2} f(\theta) + \mathrm{d}\eta_t \tag{2-15}$$

其中$\mathrm{d}\eta_t$表示符合标准布朗运动的随机扰动。

使用计算机模拟郎之万动力学系统时，需要对差分进行离散化，使用小的离散步长ϵ近似估计 2-15 式中的差分，即：

$$\theta_{t+1} \leftarrow \theta_t + \frac{\epsilon}{2} f(\theta_t) + \eta_t,\ \ \eta_t \sim \mathcal{N}(0, \epsilon) \tag{2-16}$$

使用 2-16 式进行参数空间中状态转移，进行采样的方法称为郎之万蒙特卡洛（Langevin Monte Carlo, LMC）。这种方法在状态转移时利用了势函数的梯度信息，极大加快了马尔可夫链的收敛速度。LMC 相当于使用离散化的郎之万等式（2-16）作为 MH 中的提议分布：

$$q(\theta_{t+1}|\theta_t) = \mathcal{N}\left(\theta_{t+1}; \theta_t + \frac{\epsilon}{2} f(\theta_t), \epsilon\right) \tag{2-17}$$

若$\epsilon \to 0$，离散化带来的误差也会趋于零，有$q(\theta_{t+1}|\theta_t) = 1$恒成立，即状态转移的接受概率为 1，可以接收所有样本。结合郎之万动力学的 MCMC 方法通过对势函数梯度信息的利用，可以在高维空间中很快地收敛，同时由于状态转移接受率为 1，具有很高的采样效率，解决了 MH 难以应用于贝叶斯神经网络中后验概率分布采样的两个难点。

然而，现在的机器学习问题往往需要使用很大的训练数据集进行训练。在N很大时，$f(\theta)$的计算开销过大难以进行多次迭代。近年来，使用随机估计（Stochastic Approximation, SA）的方法计算随机梯度，代替完整梯度进行参数更新的方法在大规模机器学习问题中取得了突破性的成功。随机梯度法的思想是在训练集中随机取小的子集（mini-batch），在这个小子集上计算梯度，使用这个梯度代替整个训练数据集的梯度：

$$\tilde{f}(\theta) \triangleq -\nabla\tilde{U}(\theta) = \frac{N}{n}\sum_{i=1}^{n}\nabla_\theta \log P(y_i|\boldsymbol{x}_i,\theta) + \nabla_\theta \log P(\theta) \tag{2-18}$$

使用随机梯度$\tilde{f}(\theta)$代替 2-16 式中的完整梯度，就得到了随机梯度郎之万动力学（以下简称 SGLD）——第一种 SG-MCMC 采样方法。其参数更新公式变为：

$$\theta_{t+1} \leftarrow \theta_t + \frac{\epsilon_t}{2}\tilde{f}(\theta_t) + \eta_t,\ \ \eta_t \sim \mathcal{N}(0, \epsilon_t \mathbf{I}) \tag{2-19}$$

除了应用随机梯度以外，SGLD 和 LMC 的另一个不同在于 SGLD 中步长ϵ_t不再是一个常数，而是需要满足 Robbins-Monro 条件：

$$\sum_{t=1}^{\infty}\epsilon_t = \infty \qquad\qquad \sum_{t=1}^{\infty}\epsilon_t^2 < \infty \tag{2-20}$$

上式是由 Robbins 和 Monro 于 1951 年提出的，是随机估计方法的理论基石。左边保证了参数可以在整个参数空间中进行探寻、游历，右边则保证了参数可以最终收敛。在 2014 年 Sato 和 Nakegawa 的工作[26]中，给出了 SGLD 收敛性的详细理论证明。

在第 4 章实验部分，还会对 SGLD 的实际应用做更多讨论。接下来一节，会介绍其他几种 SG-MCMC 的原理，并通过和随机优化方法的对比，解析这些方法之间的联系和区别。

表 2.2 SGLD 算法流程[①]

算法 2 SGLD Algorithm

Input: initial position θ_0 and step size sequence ϵ_t

for $t = 0,1,\ldots,T$, **do**

Evaluate stochastic gradient $\tilde{f}(\theta) \leftarrow \frac{N}{n}\sum_{i=1}^{n}\nabla_\theta \log P(y_i|\boldsymbol{x}_i,\theta) + \nabla_\theta \log P(\theta)$

Propose $\eta_t \sim \mathcal{N}(0, \epsilon_t \mathbf{I})$

Update parameter by $\theta_{t+1} \leftarrow \theta_t + \frac{\epsilon_t}{2}\tilde{f}(\theta_t) + \eta_t$

end for

① 引用自参考文献[20]

2.2.4 其他 SG-MCMC 采样方法

近年来，在 SGLD 方法的基础上，又发展出了很多新的 SG-MCMC 采样方法。在 1.1.3 节中已经对这些新 SG-MCMC 采样方法之间的关系进行过简述，在本节中，将主要从 SG-MCMC 方法和随机优化方法（Stochastic Optimization, SO）的关系。

在经典神经网络中，训练的目标不是在网络参数的后验概率分布中进行采样，而是解决 2-6 式的优化问题。然而，贝叶斯神经网络的势函数$U(\theta)$，其实就对应经典神经网络训练时被优化的目标函数，两者虽然具有不同的物理意义，但形式完全相同。因此上一节中对于贝叶斯神经网络中随机梯度的定义，可以直接沿用至随机优化方法中。

随机梯度下降法（Stochastic Gradient Descent, SGD）是在神经网络训练中取得广泛应用的最简单的一种随机优化方法。其思路就是通过在训练集的随机子集上计算梯度，使用随机梯度进行参数更新，更新公式为：

$$\theta_{t+1} \leftarrow \theta_t + \epsilon_t \tilde{f}(\theta_t) \tag{2-21}$$

对比 SGLD 的更新公式[①]：

$$\theta_{t+1} \leftarrow \theta_t + \epsilon_t \tilde{f}(\theta_t) + \eta_t,\ \ \eta_t \sim \mathcal{N}\left(0, \sqrt{2\epsilon_t}\mathbf{I}\right) \tag{2-22}$$

可以看出，SGLD 只是在 SGD 方法的基础之上，仅仅多加了一个高斯噪声，就完成了从优化方法到采样方法的转化。

在随机梯度下降法的基础上，还有很多其他改进方法：

如添加动量的随机下降法（SGD with momentum），累积动量代替梯度进行参数更新：

$$\begin{aligned} m_{t+1} &\leftarrow \mu m_t - \tilde{f}(\theta_t) \\ \theta_{t+1} &\leftarrow \theta_t - \epsilon_t m_{t+1} \end{aligned} \tag{2-23}$$

其中μ是动量因子，m_t代表累积的动量。在优化过程的初期，添加动量能加快参数的收敛速度，迅速接近极值，而优化中后期，动量则有助于跳出局部极值点，

---

① 这里将步长ϵ_t取为了 2-19 式中的 2 倍，对于算法本身没有影响，只是为了更容易看出 SGLD 和 SGD 方法之间的对应关系。

找到更好的网络参数。在 SG-MCMC 采样方法中，和添加动量的随机梯度下降法相对应的方法是随机梯度哈密尔顿动力学（SGHMC）方法。哈密尔顿动力学系统又称为二阶郎之万动力学系统中，是含有动量项和参数状态两个变量的动力学系统。

RMSprop 方法是另一种随机优化方法，其原理是在 SGD 的基础上，将每一次迭代中的随机梯度的均方根，作为学习速率的一个约束项，可以根据参数在不同维度的尺度，自动调节学习速率。在 SG-MCMC 采样方法中，也有对应的版本，称为添加预处理条件（precondition）的郎之万动力学（pSGLD）方法。pSGLD 通过计算一个预处理条件矩阵，可以根据参数在不同维度的尺度调节采样的步长，是借鉴 RMSprop 思想对 SGLD 方法进行改良。

很多 SG-MCMC 采样方法的研究受到了随机优化方法研究的启发与影响，但也有的研究者反其道而行之，使用 SG-MCMC 采样的思想对原有的优化方法进行创新。SANTA 方法是一种较为复杂的 SG-MCMC 采样方法，其随机优化版本是 Adam——本质上是添加动量项的 RMSprop。SANTA 方法分为两个阶段，第一阶段称为探寻阶段（exploration），使用 SG-MCMC 采样的思想添加噪声，对参数空间进行充分游历与探寻。之后，进入第二阶段，通过模拟退火（Temperature Annealing），将噪声的方差逐渐减小为 0，从采样过程变为优化过程，有助于得到更好的优化结果。

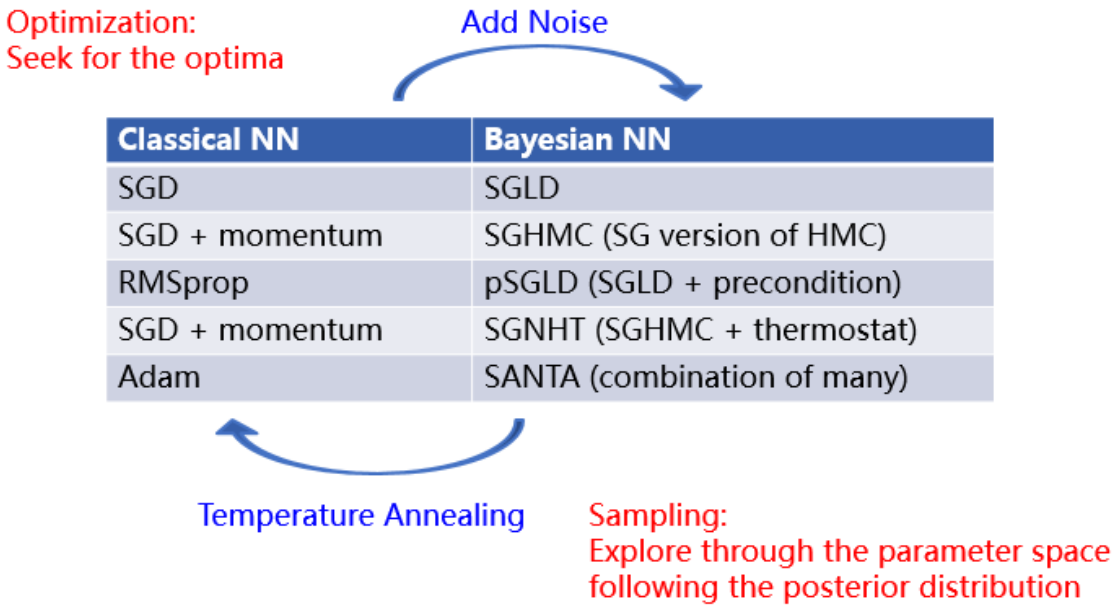

图 2.2　SG-MCMC 采样方法和随机优化方法的关系总结

图 2.1 中是我总结的 SG-MCMC 采样方法和相应的随机优化方法之间的对应关系，结合以上的分析，应该能对这两类方法的联系和区别有一个大致的认识。至于这些 SG-MCMC 采样方法的具体实现流程，可以参见附录 A 中总结的算法流程框图。从图 2.1 中可以看到，只需要通过添加噪声或模拟退火，即可完成两类方法的相互转化。SG-MCMC 方法由于是采样方法，其特点在于会对参数空间进行充分的游历与探寻，也就更容易跳出势函数中的局部极值点。而随机优化方法是对参数进行优化，在陷入一个局部极值点后就很难跳出，但能得到更好的优化结果。

充分了解 SG-MCMC 采样方法和随机优化方法之间的关系，有助于我们更深刻地理解神经网络的训练过程，对于两个领域的研究都有帮助。本研究中将 SG-MCMC 采样的结果进一步结合优化方法进行重新训练的想法，就是受到了 SG-MCMC 采样方法和随机优化方法之间差异的启发。

本章主要阐述了基于 SG-MCMC 采样的贝叶斯神经网络的整体框架和基本原理，并对该领域已有的理论研究成果进行了总结和分析。在下一章中，将会继续以理论分析为主，解决基于 SG-MCMC 采样的贝叶斯神经网络如何应用于实际的问题。

第3章　贝叶斯神经网络应用

3.1　参数先验概率分布的选择

选择先验概率分布是贝叶斯神经网络应用时的第一步。

先验概率分布$P(\theta)$一般有以下两种常见形式：第一种是取零均值、方差为σ_w^{-1}的高斯分布$P_{\mathcal{N}}(\theta)=\mathcal{N}(\theta;0,\sigma_w^{-1}\mathbf{I})$，第二种是零均值、尺度参数为$\sigma_w^{-1}$的拉普拉斯分布$P_{\mathcal{L}}(\theta)=\mathcal{L}(\theta;0,\sigma_w^{-1}\mathbf{I})$。两者的概率密度函数为：

$$f_{\mathcal{N}}(\theta;0,\sigma_w^{-1}\mathbf{I})=\sqrt{\frac{\sigma_w}{2\pi}}\mathrm{e}^{-\frac{\sigma_w}{2}\theta^T\theta} \tag{3-1}$$

$$f_{\mathcal{L}}(\theta;0,\sigma_w^{-1}\mathbf{I})=\frac{\sqrt{\sigma_w}}{2}\mathrm{e}^{-\sigma_w\sqrt{\theta^T\theta}} \tag{3-2}$$

贝叶斯神经网络中的先验概率分布，对应经典神经网络所优化的目标函数中的正则项（regularization）。正则项又称惩罚项，其作用是对模型参数值的大小进行约束，使学习到的模型尽量简单，有助于避免过拟合和提高模型的泛化能力。在经典神经网络中，常用的正则项有基于参数 L2 范数的 L2 正则和基于参数 L1 范数的 L1 正则两种。正则和先验的对应关系为：

$$R_{L2}(\theta)\triangleq\|\theta\|_2^2=\theta^T\theta\propto-\log P_{\mathcal{N}}(\theta) \tag{3-3}$$

$$R_{L1}(\theta)\triangleq\|\theta\|_1=\sum_i|\theta_i|\propto-\log P_{\mathcal{L}}(\theta) \tag{3-4}$$

即 L2 正则和高斯先验的负对数形式相同，L1 正则和拉普拉斯先验的负对数形式相同。在正则项在加入目标函数时，需要乘以一个正则强度（weight precision）系数λ，即对应先验分布中的尺度参数σ_w。为了方便讨论，接下来不再使用高斯先验和拉普拉斯先验的说法，而统称为 L2 正则和 L1 正则。

在神经网络的训练过程中，L2 正则和 L1 正则具有不同的效果。图 3.1 分别画出了 L2 正则和 L1 正则在一个 3 维参数空间中的等值面。从图 3.1（a）中可以看出，L2 正则在对参数大小进行限制时，是对所有参数的数值进行无差别惩罚，因此在空间中的等值面是一个以原点为中心的球体。而 L1 正则由于其一阶导数

为常数，且对于任意维度在 0 点不连续，其形状是如图 3.1（b）所示的八面体。L1 正则对于各参数值的惩罚是不同的，在八面体的边和定点存在极值点，更容易得到这些位置的参数。在边上时，3 维的参数中有 1 维为 0，而在定点时，3 维的参数中有 2 维都为 0，为模型带来了稀疏性。

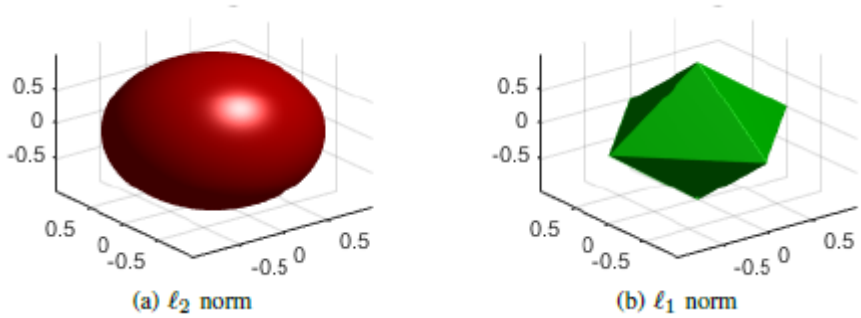

图 3.1　3 维参数空间中 L2 正则和 L1 正则等值面示意图[①]

模型参数的稀疏性是模型压缩中的重要性质。在对神经网络进行网络剪枝时中，应用两种不同的正则也得到了不同的实验结果，将在 4.3.1 节中进行讨论

3.2　应用模型

在本研究中，将基于 SG-MCMC 采样的贝叶斯学习框架应用到了两个模型之中，分别是深度神经网络（Deep Neural Network, DNN）模型和长短时记忆网络（Long Short Term Memory，LSTM）模型。这一节将简要介绍这两种模型的原理。

3.2.1　深度神经网络模型

深度神经网络模型一般指含有两个隐层及以上的全连接神经网络模型。对于一个层数为L的深度神经网络，第 0 层为输入层，第 1 到第$L-1$层为隐层，第L为输出层。设神经网络第l层的输出为$\boldsymbol{z}^l$，前一层的输出即为后一层的输入，则计算$\boldsymbol{z}^l$可由下式进行计算：

$$\boldsymbol{a}^l = \mathbf{W}^l \boldsymbol{z}^{l-1} + \boldsymbol{b}^l, \boldsymbol{z}^l = \sigma(\boldsymbol{a}^l) \tag{3-5}$$

① 图片引自参考文献[27]

其中$\mathbf{W}^l$是第$l-1$层与第l层之间的权重矩阵，$\boldsymbol{b}^l$是第l层的偏置向量，$\sigma(\cdot)$表示任意一种非线性激活函数。$\boldsymbol{z}^l$的维数即为第l层的隐层神经元数目。

在分类问题中，输出层的维度就是分类的类别数目，通过 softmax 激活函数计算各个类别的概率，选择概率最大的最为分类结果。

3.2.2 长短时记忆网络模型

在介绍长短时记忆网络前，首先要介绍一下循环神经网络（Recurrent Neural Network, RNN）的基本概念。循环神经网络是应用于序列建模的一种神经网络模型，其特点是输入可以是一个和时间有关的序列$\boldsymbol{X}=\{\boldsymbol{x}_1,\dots,\boldsymbol{x}_T\}$，其中$\boldsymbol{x}_t$是$t$时刻的输入向量。在每一个时刻$t$还有一个相应的隐藏状态$\boldsymbol{h}_t$，由当前时刻的输入$\boldsymbol{x}_t$和前一时刻的隐藏状态$\boldsymbol{h}_{t-1}$决定，即$\boldsymbol{h}_t=f(\boldsymbol{h}_{t-1},\boldsymbol{x}_t)$，$f(\cdot)$称为转移函数。RNN 的输出也是一个和时间有关的序列$\boldsymbol{Y}=\{\boldsymbol{y}_1,\dots,\boldsymbol{y}_T\}$，即完成了序列级别的建模，适用于自然语言处理、句子分类、机器翻译等需要对序列建模的问题。

LSTM 是在 RNN 的基础上发展而来的，使用各种“门”作为激活函数的一类 RNN 模型[28]。相比 RNN，LSTM 的优势在于可以处理序列的长时依赖，不会发生 RNN 中在循环迭代次数较多时会出现梯度消失的问题。LSTM 比 RNN 多一个表示当前状态的量$\boldsymbol{c}_t$，称为t时刻的细胞状态（cell state）。细胞状态可以视为 LSTM 的记忆单元，受几个 sigmoid 门控制：由$\boldsymbol{i}_t$控制从输入中加入的信息，$\boldsymbol{f}_t$控制从原细胞状态中遗忘的信息，$\boldsymbol{o}_t$控制从细胞状态输出的信息。在每次出现新的输入时，细胞状态和隐藏状态都会根据以下规则进行信息的遗忘和更新：

$$\begin{aligned}
\boldsymbol{i}_t &= \sigma(\mathbf{W}_i\boldsymbol{x}_t+\mathbf{U}_i\boldsymbol{h}_{t-1}+\boldsymbol{b}_i)\\
\boldsymbol{f}_t &= \sigma(\mathbf{W}_f\boldsymbol{x}_t+\mathbf{U}_f\boldsymbol{h}_{t-1}+\boldsymbol{b}_f)\\
\boldsymbol{o}_t &= \sigma(\mathbf{W}_o\boldsymbol{x}_t+\mathbf{U}_o\boldsymbol{h}_{t-1}+\boldsymbol{b}_o)\\
\tilde{\boldsymbol{c}}_t &= \tanh(\mathbf{W}_c\boldsymbol{x}_t+\mathbf{U}_c\boldsymbol{h}_{t-1}+\boldsymbol{b}_c)\\
\boldsymbol{c}_t &= \boldsymbol{f}_t\odot\boldsymbol{c}_{t-1}+\boldsymbol{i}_t\odot\tilde{\boldsymbol{c}}_t\\
\boldsymbol{h}_t &= \boldsymbol{o}_t\odot\tanh(\boldsymbol{c}_t)
\end{aligned} \tag{3-6}$$

其中$\sigma(\cdot)$表示 sigmoid 激活函数，$\odot$表示逐元素点积，$\mathbf{W}$、$\mathbf{U}$、$\boldsymbol{b}$分别为编码权重矩阵，循环权重矩阵和偏置项。LSTM 也可以通过叠加称为深层模型，叠加的方法是将上一层 LSTM 的隐藏状态作为下一层 LSTM 的输入。

3.3 针对应用难点的解决方案

基于 SG-MCMC 采样的贝叶斯神经网络在应用时的主要难点，在于推断时需要进行模型平均，在 1.2.2 节中已经对这一点进行过分析。模型平均意味着成倍增加的模型存储空间开销和预测计算时间开销，然而在现实应用中，为了在复杂任务中取得更好的表现，模型的参数规模往往十分庞大，无法负担起高额的存储成本。在推断时，预测所需时间的长短也直接关系到模型能否进行实际应用，运算时间过长会极大影响用户体验。

下面两节，就将针对这两点问题，给出基于网络剪枝的模型压缩和网络参数重训练这两个解决方案。

3.3.1 基于网络剪枝进行模型压缩的原理

在神经网络模型中有大量的参数，也存在着大量的冗余，如何进行模型压缩从而减轻庞大模型的存储负担，是目前深度学习领域的一大研究问题。基于网络剪枝的模型压缩原理非常简单，就是在训练神经网络的过程中，将网络中的一些连接砍掉，减小模型的规模。在 Han 的工作[29]中，通过迭代进行网络剪枝和剪枝之后的重训练，可以将深度神经网络的模型规模压缩 12 倍，即存储代价降低到原来的 1/12，而不损失预测准确度。

在神经网络中，权重值小的连接，对于网络输出的贡献就少，可以被认为是“不重要”的连接。当这些连接被剪掉时，由于本身权重较小，剪枝即权重值置 0 后，对网络输出的影响也不大。基于这点认识，可以设置某一剪枝阈值，对于小于这一阈值的网络连接进行剪枝。

网络剪枝方法也可以在贝叶斯神经网络中进行应用。在通过 SG-MCMC 采样得到网络参数的样本后，对每一个样本都进行一次网络剪枝，存储剪枝过的模型，之后再使用这种经过剪枝压缩的模型进行推断。由于在采样过程中，网络参数的大小一直在变化，难以找到一个固定的阈值进行剪枝。在实际应用时，可以只设置一个剪枝比例，按比例动态计算剪枝阈值。具体的算法流程见表 3.1。

在网络剪枝过后，模型的存储开销成倍降低，可以抵消存储多个模型参数样本带来的存储量成倍增大的问题。有关贝叶斯神经网络剪枝的具体实验结果，参见第 4 章的 4.3.1 节。

表 3.1 网络剪枝流程

算法 3 贝叶斯神经网络剪枝算法

输入：剪枝比例η

训练过程：

对于$i = 1,2,\dots,n$：

1. 使用 SG-MCMC 采样得到第 i个网络参数样本θ_i
2. 将θ_i的值按绝对值从小到大排序得到S_i，$i = 1,2,\dots,d$，d为θ的维数
3. $k = \mathrm{round}(\eta d)$，剪枝阈值$\lambda = S_k$
4. 根据阈值进行网络剪枝
5. 存储剪枝后的样本θ_i^{pruned}

推断过程：

使用剪枝过后的样本进行推断：$P(y^*|\boldsymbol{x}^*,\mathcal{D}) = \frac{1}{n}\sum_{i=1}^{n} P\left(y^*\middle|\boldsymbol{x}^*,\theta_i^{pruned}\right)$

3.3.2 网络参数重训练

模型压缩使得模型规模减小，可以加快单个模型预测时的速度，在一定程度上缓解贝叶斯神经网络推断时间过长的问题。但由于仍然需要进行多模型的平均，模型压缩带来的推断时间减少收效甚微。事实上，只要使用蒙特卡洛积分来近似预测分布的积分，就一定存在模型平均，难以从根本上解决问题。

网络参数重训练的初衷就是为了避免进行模型平均。其原理很简单，就是对于贝叶斯神经网络采样得到的多个网络参数样本，对每一组网络参数都使用随机优化的方法进行重新训练。重训练完成之后，只选择一个最优的模型，作为最终的模型，进行推断。具体的算法流程见表 3.2。

这种方法的思路在于，由于 SG-MCMC 采样方法可以对神经网络的参数空间进行充分的探寻，采样得到的样本中具有进一步优化找到更好的极值点的潜力，那么直接用重训练优化后的单一模型的预测结果，未必会比之前模型平均的结果要差。实验证明，经过参数重训练后选择出的最优模型，确实可以得到比模型平均更好的预测结果。并且，重训练还可以和网络剪枝同时应用，有助于模型规模的进一步压缩。具体的实验结果，参见第 4 章的 4.3.2 节。

表 3.2 网络参数重训练流程

算法 4 网络参数采样-重训练算法

训练过程：

对于$i = 1,2,\dots,n$：

1. 使用 SG-MCMC 采样得到第 i个网络参数样本θ_i
2. 若进行网络剪枝，则按表 3.1 的流程进行剪枝，否则跳过此步骤
3. 使用随机优化的方法对θ_i进行重训练

推断过程：

基于验证集选择参数重训练后的最优模型θ^*

使用θ^*进行推断：$P(y^*|\boldsymbol{x}^*,\mathcal{D}) = P(y^*|\boldsymbol{x}^*,\theta^*)$

原则上讲，网络参数重训练已经脱离了贝叶斯神经网络的框架，成为了一种独立的网络参数优化方法。但由于在现有基于采样的贝叶斯神经网络框架中，多个模型参数也仅仅是用来进行预测后取平均，并没有利用多个预测结果可以提供的其它统计信息，和使用单一模型进行预测并没有本质区别。因此，在推断时使用网络参数重训练后的最优模型作为贝叶斯方法中模型平均的替代，也是目前来看很有意义的一种解决方案。

第4章　实验结果

本章将会分为三节介绍本研究中得到的实验结果。4.1 节是对实验环境和工程实现的简要介绍，然后在接下来的两节里分别介绍验证性实验结果和创新性实验结果这两方面工作。

4.1 实验基本情况

4.1.1 实验环境

本研究中的所有实验均在以下环境中实现：

表 4.1 实验环境

实验环境	具体情况
操作系统	CentOS Linux 7
处理器	Inter® Xeon(R) CPU E5-2690 v4 @ 2.60GHz x56
4 块 GPU①	GeForce GTX 1080
显存大小	8.00GB
编程语言	Python 3.4
深度学习平台	Tensorflow② 1.0.0

4.1.2 工程实现

基于 Tensorflow 实现了基于梯度的 MCMC 采样的标准方法 HMC 方法和 4 种 SG-MCMC 采样方法：SGLD，SGHMC，pSGLD 和 SANTA。

在编程实现时，我采用了层次化编程的方式，类之间的继承与派生关系如图 4.2 所示。定义基类 Sampler 用于进行输入输出数据的规一化等一些基本函数，然后在 Sampler 类的基础上派生出 MCMC sampler 类，在该类中实现模型构建，通过当前网络参数计算似然函数、先验概率、后验概率及其梯度，采样，推断核心

① 未进行分布式编程，所有实验仅使用 1 块 GPU 运行

② TensorFlow™ 是由 Google 公司发布的一个采用数据流图（data flow graphs），用于数值计算的开源软件库，在机器学习和深度神经网络方面的研究中具有广泛应用[30]。

方法。然后在 MCMC sampler 的基础上，根据各采样算法的流程，再派生出各个采样方法的子类，在子类中实现不同的状态转移方法。这种层次化编程的好处是结构清晰，实现基本功能的代码具有很高的重用性，且各采样方法之间不会互相影响，便于调试研究。

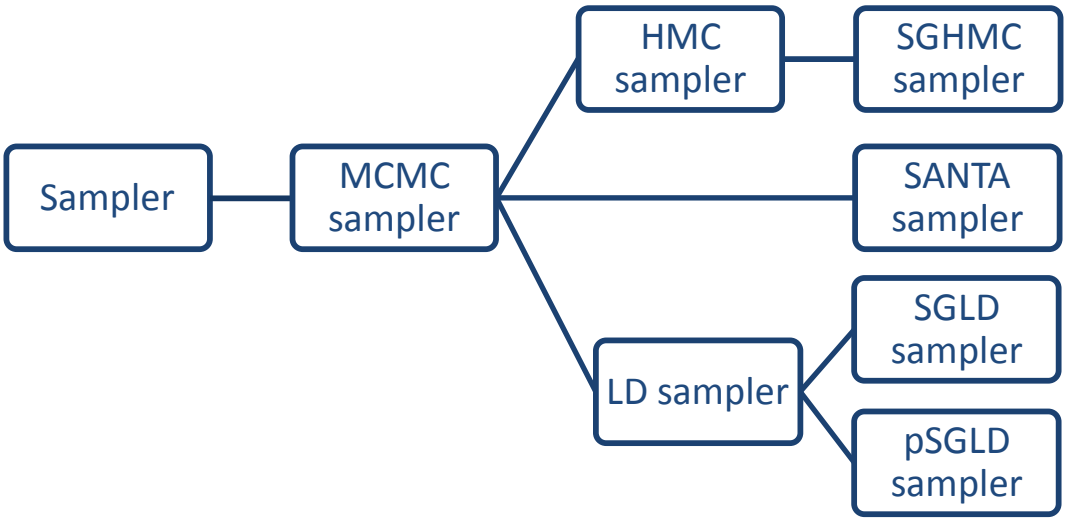

图 4.1 层次化编程结构示意图

随机优化方法中，使用 SGD 和 Adam 方法作为实验的基准结果，以便和贝叶斯神经网络得到的结果进行对比。这两种方法可以直接通过调用 Tensorflow 封装好的 GradientDescentOptimizer 和 AdamOptimizer 方法实现。

模型方面，实现了深度神经网络和 LSTM 网络两类模型，分别完成手写体识别和语言模型建模任务。由于这两种模型中的常用函数已经在 Tensorflow 中封装好，实际使用时只需要定义模型的结构和连接关系就行，十分方便。

至于具体的模型结构，采样、优化方法中超参数的设置，以及基于网络剪枝的模型压缩和网络参数重训练的相关实验设置，会在对应的实验部分进行说明。

4.2 验证性实验结果

验证性实验是指针对目前基于 SG-MCMC 采样的贝叶斯神经网络研究领域中已有的理论或方法进行复现实验，验证实验结果是否符合理论预期或者能否复现他人的研究结果。

本研究中一共进行了四个验证性实验，前两个仿真实验用来测试 SG-MCMC 采样方法的性能，并进行几种方法采样性能的对比。之后选择了性能较好的两种方法 SGLD 和 SANTA，在两个实际问题中进行了应用，得到了基于 SG-MCMC 采样的贝叶斯神经网络和经典神经网络的对比结果。

4.2.1 对给定分布进行采样

仿照参考文献[17]中的实验，对一个概率密度函数如图 4.2 所示的一维双峰分布进行采样，通过采到的样本估计这个概率密度。该分布的势函数为：

$$U(\theta)=(\theta+4)(\theta+1)(\theta-1)(\theta-3)/14+0.5 \tag{4-1}$$

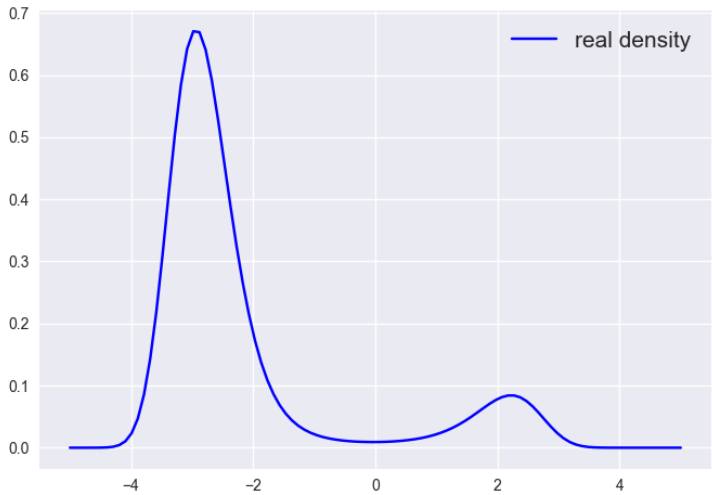

图 4.2 采样实验中的真实概率密度函数

通过绘制样本的统计分布直方图，可以直观地看出采样方法对真实概率分布的估计情况，进而比较各种采样方法的对于这个给定分布的采样性能。采样时，将马尔可夫链历经的前 5000 个样本作为收敛的过程（burn in）而丢弃掉，分别取

之后的 10000 或 3000 个样本进行估计时，结果如下：

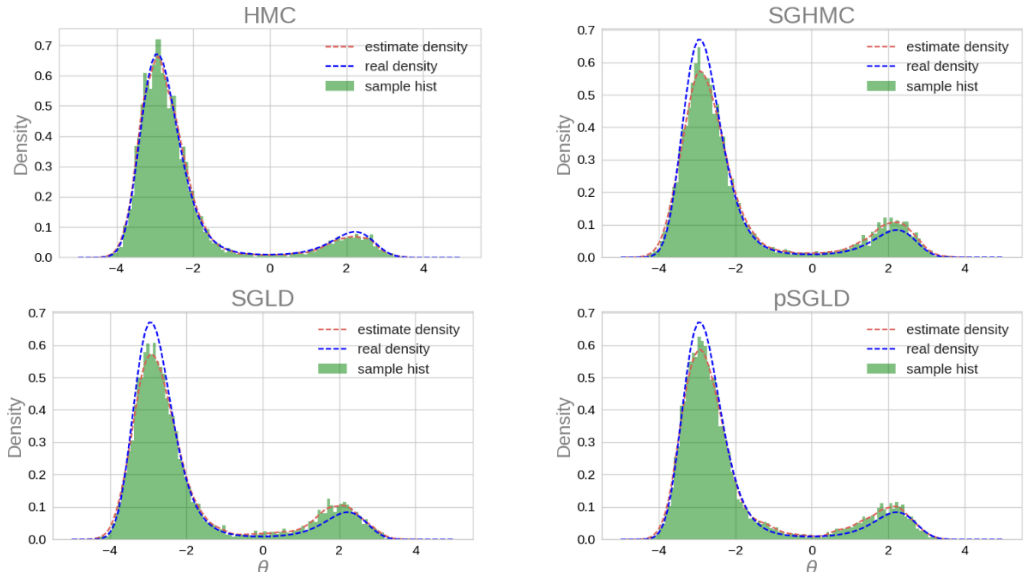

图 4.3 采样 10000 个样本时的估计结果

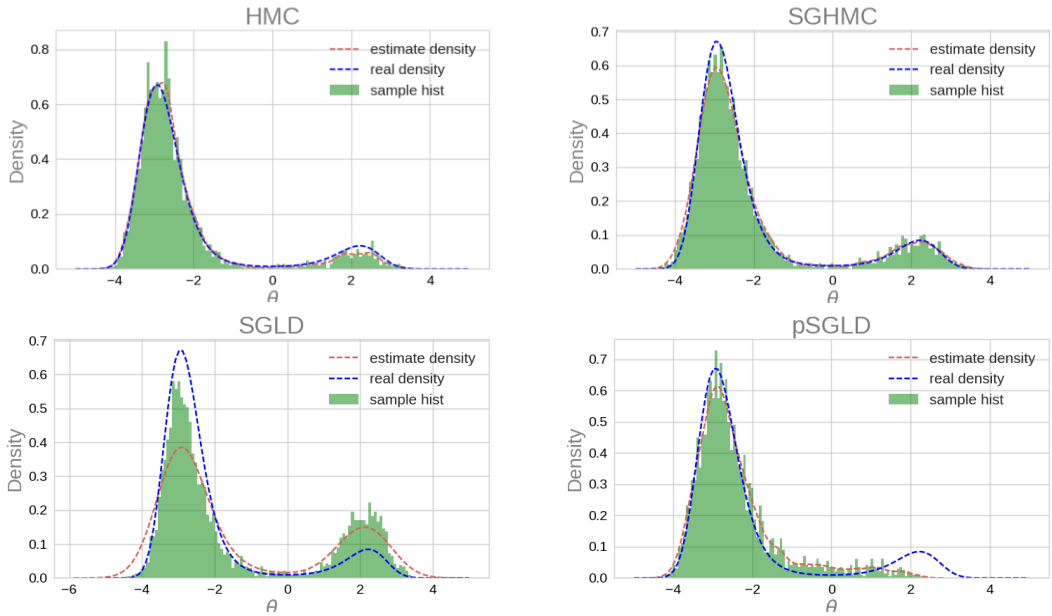

图 4.4 采样 3000 个样本时的估计结果

从图 4.3 的结果可以看出，四种采样方法都能够比较准确的估计出真实的概率分布，证明了 SG-MCMC 采样方法的有效性。若使用样本分布和真实分布之间的KL距离进行定量分析，则有$\mathrm{KL}_{\mathrm{HMC}} < \mathrm{KL}_{\mathrm{pSGLD}} < \mathrm{KL}_{\mathrm{SGLD}} < \mathrm{KL}_{\mathrm{SGHMC}}$，KL 距离越小的表明两个分布相似度越高，采样性能越好。

然而，当采样的样本数减少为 3000 个时，估计结果就产生了比较大的差异：从图 4.4 中可以看到，HMC 和 SGHMC 仍旧做出了十分准确的估计，然而 SGLD 和 pSGLD 的估计就没有那么准确了，SGLD 在两个概率峰之间采到的样本数量比例不对，而 pSGLD 则基本没有采到概率密度较低的峰的样本。通过两种方法的原理进行分析，出现这种现象的原因在于步长的选择。对于 SGLD，出现估计偏差是由于步长取值过大，离散化带来的误差已不能被忽略所致，具体表现就是参数在空间中跳转得太远。而对于 pSGLD，则是步长取值过小，导致样本的相关性过高，难以在空间中进行有效探寻。在分布调小、调大 SGLD 和 pSGLD 的步长后，两种方法也能得到和图 4.3 中相似的估计结果，验证了之前的分析。

该实验的结论是进行对比的三种 SG-MCMC 采样方法和基准方法 HMC 方法，都可以在这个简单的一维分布采样问题中进行有效采样，初步验证了这些方法的有效性。因为该方法

4.2.2 曲线回归

4.1 中的采样实验针对的是采样方法本身，在本节中，将结合神经网络模型，考察 4.1 节中的四种方法在$y = x^3$曲线回归问题中的应用。

本实验使用随机生成的仿真数据。在$[-4, 4]$区间随机生成 20 个样本点$x_1, \ldots x_{20}$，计算 $y = x^3 + \mathcal{N}(0,9)$，组成训练集$\mathcal{D} = \{(x_i,\ y_i)\},\ i = 1, \ldots, 20$。使用这些训练样本分别对采用 HMC，SGHMC， SGLD，pSGLD 方法进行网络参数后验概率分布采样的贝叶斯神经网络进行训练，得到模型参数的若干后验样本。在推断时，对于每一个测试输入，计算神经网络输出在所有后验样本上输出的平均值和标准差，作为预测结果和结果的置信区间。

实验的具体设置为：使用一个隐层，100 个隐层神经元的全连接神经网络。在进行训练时，对 20 个训练数据循环训练 10000 个迭代周期，前 4500 个迭代周期作为 burn in 过程，将采到的样本丢弃，只保存最后 5500 个样本在预测时进行平均。对于三种 SG-MCMC 方法，每次梯度计算使用随机选取的包含 5 个数据的小子集进行，而 HMC 则使用整个训练集的 20 个数据计算梯度。各个方法的超参数取值为：HMC 的步长$\epsilon = 5\text{e-}4$，每次更新参数的转移步数$\mathrm{L} = 10$；SGHMC 汇总的步长$\epsilon = 5\text{e-}3,\ \mathrm{L} = 10$；SGLD 中步长$\epsilon$ =1.25e-5；pSGLD 中步长ϵ =5e-5，$\lambda = 0.1$，$\beta = 0.99$。是通过重复多次实验进行尝试，比较拟合误差的方法所选择出的最优参数。

四种方法的曲线回归结果如图 4.5-4.8 所示：

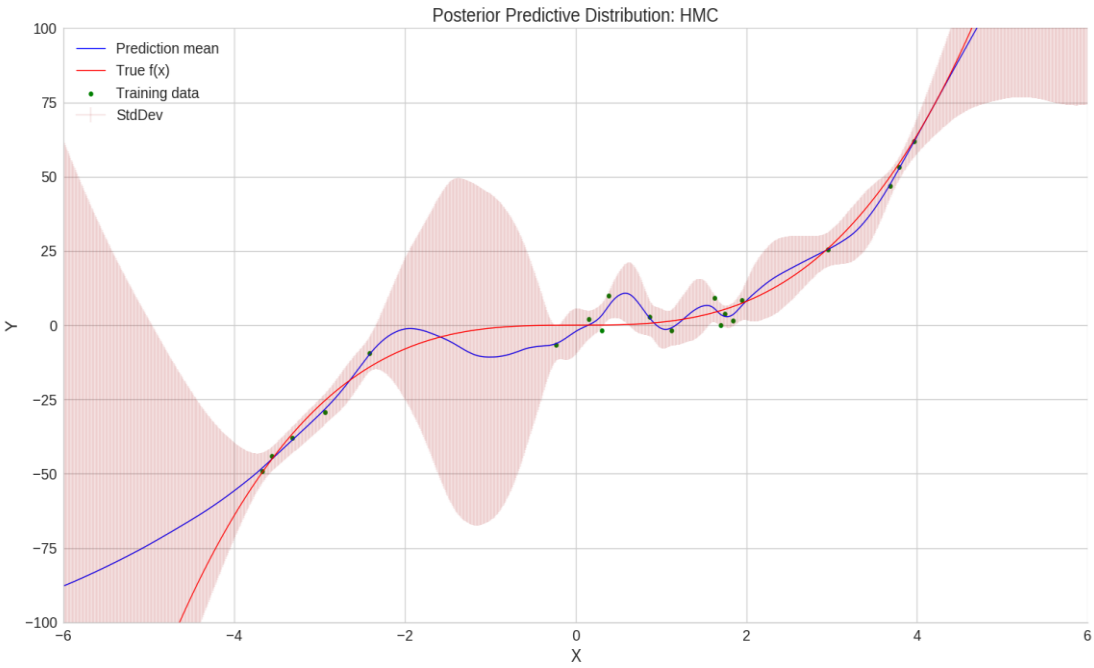

图 4.5 HMC 的曲线回归结果

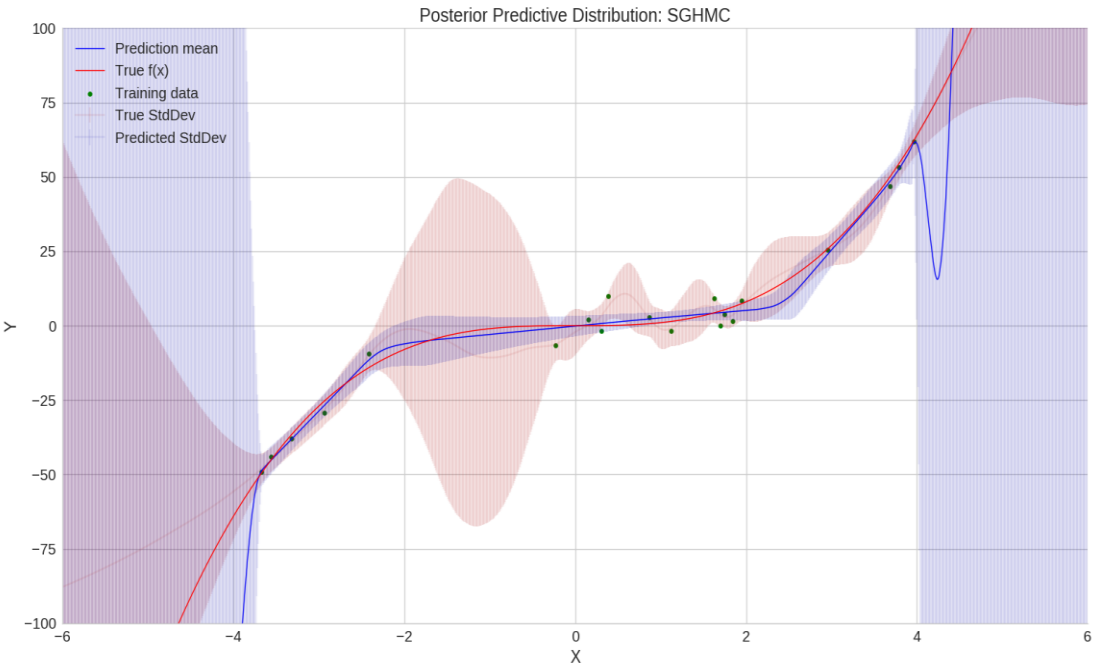

图 4.6 SGHMC 的曲线回归结果

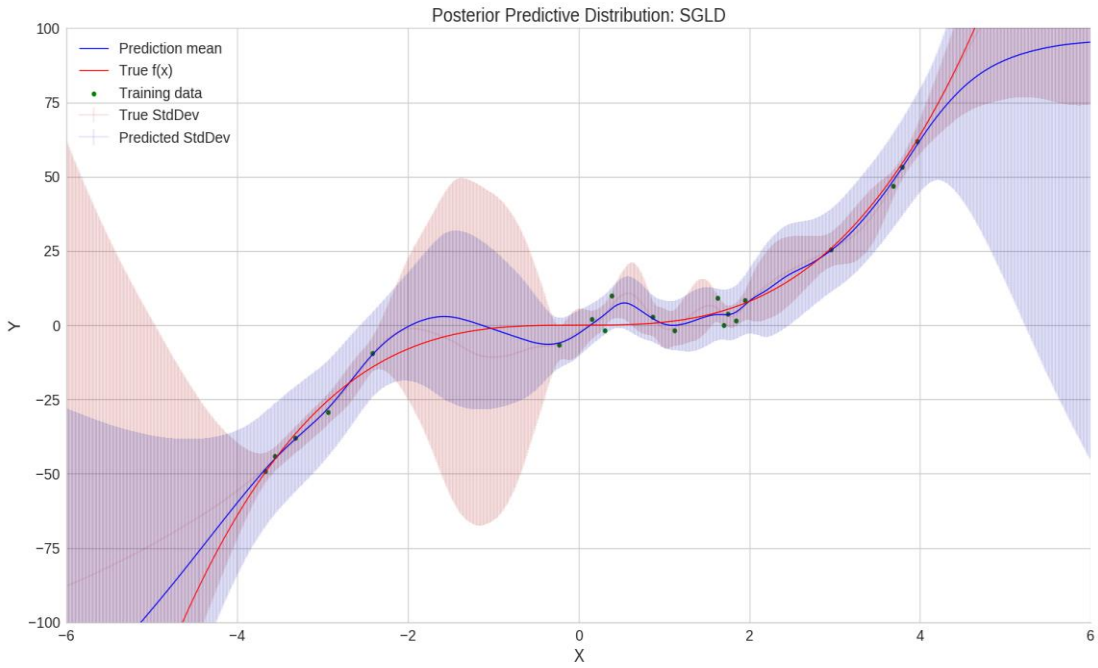

图 4.7 SGLD 的曲线回归拟合结果

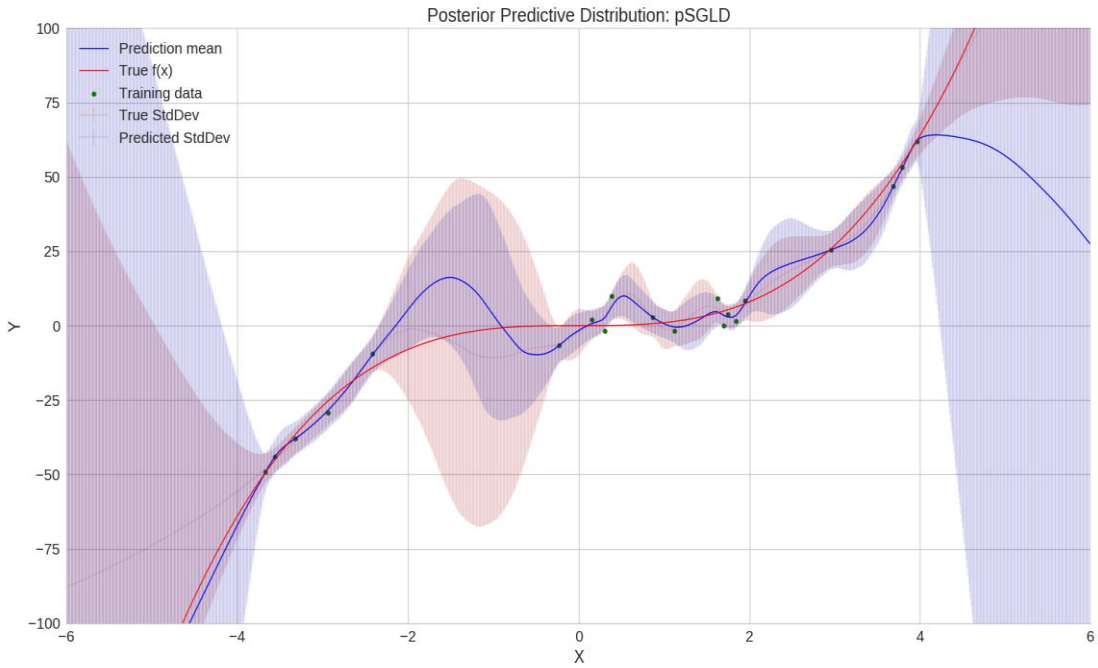

图 4.8 pSGLD 的曲线回归拟合结果

在上面的归回结果图中，红色线代表真实的$y = x^3$曲线，黑点代表训练数据点，蓝色线是贝叶斯神经网络使用所有样本进行预测的平均值。图 4.5 是 HMC 的结果，红色阴影区间代表的是预测平均值加减一倍标准差，在图 4.6-4.8 中也画出了 HMC 拟合的均值和一倍标准差区间，作为其他三种 SG-MCMC 方法的基准，即图例中的“True StdDev”。在后面三张图中，预测的一倍标准差区间用蓝色阴影区间表示，即图例中的“Predicted StdDev”，以便和 HMC 的基准结果区分。

从 HMC 方法的拟合结果可以看出，一倍标准差区间作为贝叶斯神经网络不确定度的度量，比较好地反映了预测结果的置信度：直观来看，在训练数据密集的地方，模型对自己的预测更为自信，标准差区间小；而在[-2,0]这种训练数据较少的区间，预测结果的标准差很大，表明模型对于这个区间内数据的预测结果不太确定，置信度低。用这种直观标准衡量 SGHMC，SGLD 和 pSGLD，可以看出 SGHMC 虽然对于预测结果均值的拟合基本正确，但得到的不确定度随着训练数据疏密的改变变化不大，难以起到评估不确定度的作用。而 SGLD 和 pSGLD 则得到了和 HMC 相似的，可以提供有效不确定度度量的置信区间，且从拟合结果来看，最为简单、调参最为容易的 SGLD 反而得到了最好的曲线拟合结果，在-6 到 6 的测试区间上取得了最小的均方根误差。

曲线回归中的标准差区间是贝叶斯神经网络的不确定性度量中最为直观的体现。然而，虽然能在拟合结果图中看出不确定度的意义，但仍然难以对其进行定量估计，定量比较不同方法所提供的不确定度的优劣。通过下游任务评价贝叶斯神经网络不确定度的困难在 1.2.1 节中已经讨论过，因此，本论文没有对不确定度的度量进行更加深入的研究。

本节实验的意义，在于对贝叶斯神经网络所能提供的不确定度量进行一种直观展示，同时通过三种 SG-MCMC 采样方法的拟合结果的比较，确定了性能最好、也最适合进行调试研究的方法 SGLD。在后续研究中，由于研究重点从 SG-MCMC 方法的对比转变为贝叶斯神经网络和传统神经网络优化方法的对比，将只使用 SGLD 作为 SG-MCMC 采样方法的实验结果。

4.2.3 基于 DNN 的 MNIST 手写体识别

MNIST 数据集是一个手写数字的图片数据库，含有 70000 张 28*28 大小的阿拉伯数字 0-9 的手写体图像。训练集包括 60000 幅图像，测试集包括 10000 副图像，很多研究工作都会给出在 MNIST 数据集上进行模型训练和测试的结果，是图像识别领域中被研究最多的任务之一。

本实验使用有两个隐层，每个隐层分别含有 300，100 个隐层神经元的深度神经网络模型，该模型也被称为 Lenet-300-100，是基于 MNIST 数据集的研究中经常使用的一种神经网络结构。两个隐层选择 ReLU 作为激活函数，输出层使用 Softmax 激活函数计算各类别的概率。

贝叶斯神经网络在训练时的细节如下：每次迭代取 100 个训练数据作为 mini-batch 计算随机梯度；当使用整个训练数据集循环训练 100 次，即进行 60000 次迭代时停止训练；前 20 次循环作为 SG-MCMC 方法收敛的过程，不进行采样，在之后 80 次循环暨 48000 次迭代的过程中，每 2000 次迭代采样一次，一共得到 24 个网络参数样本。

使用四种方法进行了实验，分别是 SGLD，SANTA 这两种 SG-MCMC 采样方法和分别和对应的随机优化方法 SGD，Adam 进行对比。各方法的细节如下：SGLD 的步长$\epsilon = 0.5$，使用正则强度$\lambda = 25$的 L2 正则；SANTA 的步长$\epsilon = 4\text{e-}6$，且训练集每 20 次循环衰减 0.5 倍，模拟退火的速率$\gamma = 0.7$，其他参数使用论文[20]算法的默认设置，使用正则强度$\lambda = 5$的 L2 正则。SGD 和 Adam 的学习速率ϵ分别取 0.5 和$5\text{e-}6$，使用正则强度$\lambda = 1$的 L2 正则，同样使用训练数据循环训练 100 个周期。

以上实验参数的设置均为取不同值进行多次实验，得到的最好实验结果。四种方法在 MNIST 测试集上的识别错误率如下表所示：

表 4.2 MNIST 测试错误率对比

方法	所属类别	测试错误率
SGD	随机优化	1.66%±0.03%
Adam	随机优化	1.61%±0.01%
SGLD	SG-MCMC 采样	1.43%±0.07%
SANTA	SG-MCMC 采样	1.37%±0.04%

为消除实验随机性的影响，所有实验均独立运行 10 次，取 10 次实验的平均测试错误率加减一倍标准差作为该方法的实验结果。其中，SGLD 和 SANTA 的结果是使用采样得到的 24 个网络参数样本进行预测后平均的结果。

Adam 方法得到的 1.61%识别错误率基本上是随机优化方法在上文中所述的实验条件下能够得到的最好结果。而两种基于 SG-MCMC 采样的贝叶斯方法则可

以比 Adam 方法的识别错误率再分别下降 0.18%和 0.24%，相对降低 11%和 15%，得到了十分可观的表现提升。然而，由于贝叶斯神经网络的结果是是用 24 个模型平均得到的，所需要的存储空间和预测时间都是随机优化方法的 24 倍，以 SANTA 方法为例，表明错误率每下降 1%需要多消耗 1.6 倍的资源，性能提升的代价非常昂贵。

4.2.4 基于 LSTM 的语言模型建模

语言模型是表征一个语句出现概率的统计模型，在语音识别领域常常被用于基于一个句子中给定的一串词计算下一个词的概率分布，对接下来最可能出现的词做出预测[①]。

使用 LSTM 网络进行语言模型的建模时，模型训练的输入数据是一个个长度为T的词序列$\boldsymbol{x}_1, \boldsymbol{x}_2, \dots, \boldsymbol{x}_T$，对于$t = 1,2, \dots, T-1$，训练的标签为$\boldsymbol{y}_t = \boldsymbol{x}_{t+1}$，即训练 LSTM 对当前时刻输入的词之后的一个词进行预测。

PTB（Penn TreeBank）数据集是目前在语言模型研究中应用最为广泛的数据集之一[32]。PTB 包括已经划分好的训练集、验证机和测试集，在训练集中就有包括 10000 种单词的近 90 万个词语。单词的种类又称为一个数据集的词表大小，记为 N。

评价一个语言模型好坏的标准称为混淆度（Perplexity），其计算公式如下：

$$PPL = \exp(-\frac{1}{N}\sum_i P(w_i)) \tag{4-1}$$

其中$P(w_i)$是模型对于测试集中第i个词的预测概率。混淆度可以直观理解为预测下一个词时有可能成为该词的候选词数目，因此混淆度越小的语言模型性能越好。

本实验使用两层，每层 200 个隐层神经元，循环展开数目为 20 的小型 LSTM 模型进行研究。训练时长设定为使用所有训练数据循环训练 30 个周期，其中前 10 次循环作为 SGLD 方法收敛的过程，不进行采样，之后每一个大循环采样一个网络参数样本，一共得到 20 个网络参数样本。随机优化方法则使用最基本的 SGD 方法作为基准结果，实验结果如表 4.3 所示：

---

① 本研究中只考虑词语级别的语言模型建模问题。

表 4.3 基于 PTB 数据集的语言模型建模实验结果对比

	SGD	SGLD
测试集 ppl	114.2	101.6
训练耗时（秒/周期）	55	58
测试耗时（秒）	160	160*20=3200

SGLD 比 SGD 的测试集混淆度降低了 12.6，相对降低 11%，提升明显。但是由于使用了 20 个模型进行平均，SGLD 在推断过程中进行测试的耗时直接提高到了 SGD 的 20 倍，虽然有混淆度上的提升，却无法进行实际应用。

4.3 创新性实验结果

从上一节贝叶斯神经网络在手写体识别和语言模型建模任务中的实验结果可以看出，贝叶斯方法有助于提高模型做出正确预测的能力，但需要付出十分昂贵的代价。本节中，将使用 SGLD 方法，在 4.2.3 中所述的实验条件下进行更深入的研究，给出基于网络剪枝的模型压缩和网络参数重训练两种方法的实验结果，根据实验结果分析这两种方法如何解决贝叶斯神经网络应用中的难题。

4.3.1 基于网络剪枝的模型压缩

贝叶斯神经网络剪枝实验按照表 3.1 中的流程进行。

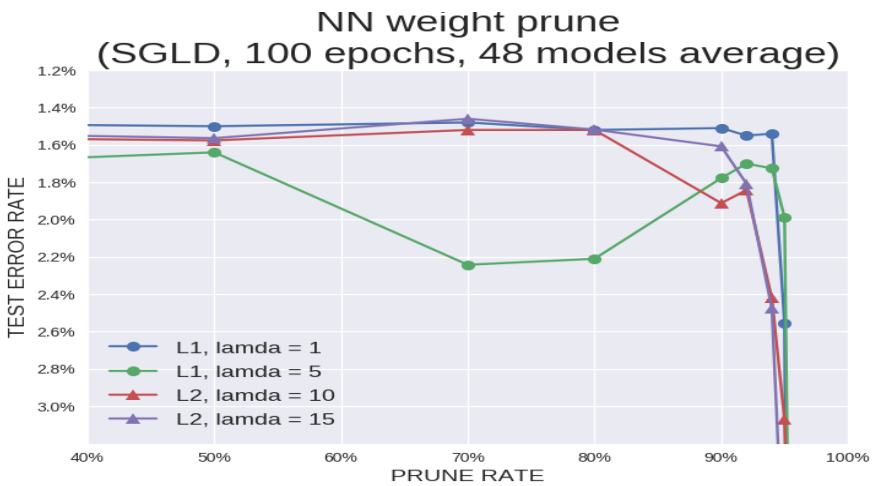

图 4.9 不同正则形式、正则强度下使用 SGLD 方法的网络剪枝结果

图 4.9 给出了不同正则类型、正则强度下，当剪枝比例从 40%上升到 95%以上时，测试错误率随剪枝比例变化的趋势曲线。得到的结果令人惊叹：使用 L1 正则，正则强度$\lambda = 1$时，SGLD 可以在剪枝 90%以上时，仍然达到 1.51%的测试错误率。在不同正则的对比中，L1 正则在进行网络压缩时展现出了更好的性能，因为其对网络参数具有稀疏性约束，采样得到的网络参数样本稀疏性高，在剪枝后对不影响网络输出的可能性更大。

单纯的讨论网络剪枝比例是没有意义的，关键在于如何在网络剪枝后还能保持模型的预测准确度。表 4.4 中给出了在测试错误率的相对损失低于 2%的前提条件下，可能达到的最大网络剪枝比例：

表 4.4 网络剪枝实验结果

方法	方法细节	存储空间	推断时间	测试错误率
Adam	基准结果	1 倍	1 倍	1.61%±0.01%
Adam+剪枝	剪枝比例 50%	0.5 倍	1 倍	1.63%±0.13%
SGLD	24 个模型进行平均	24 倍	24 倍	1.43%±0.07%
SGLD+剪枝	24 个模型进行平均 每个模型剪枝比例 92%	1.9 倍	24 倍	1.43%±0.12%

表 4.4 中的测试错误率仍然是独立运行 10 次实验的结果。可以看出，在进行权重剪枝后，由于没有重训练进行调整，Adam 方法只能剪掉一半的网络连接，使网络参数规模压缩 2 倍。而 SGLD 由于在权重剪枝后还会对预测结果进行平均，可以在很大程度上弥补权重剪枝带来的预测准确率损失，使得模型平均后的权重剪枝比例可以高达 92%，仍然不会损失准确率。剪掉 92%的连接意味着每一个模型被压缩了 12.5 倍，存储 24 个模型也只需要不到 2 倍的存储空间开销，并且能带来 12%的测试错误率相对降低。

在进行模型剪枝后，SGLD 的模型存储开销可以降低到正常模型的 2 倍大小，但其在测试错误率上的提升却没有因此打折扣，从存储意义上解决了贝叶斯神经网络的应用难题。但是，剪枝带来的模型规模的减小只能在很小程度上减少模型的推断时间，由于剪枝后模型的数目没变，推断时的庞大时间开销仍然制约着结合网络剪枝的贝叶斯神经网络的实际应用。

4.3.2 网络参数重训练

网络参数重训练的实验按表 3.1 中的流程进行。在实验中，为保证训练时长相同，进行网络参数重训练时，SGLD 采样过程减少为 60 个训练集循环周期，再用 20 个训练集循环周期进行参数重训练，因此采样的到的网络参数样本数由 24 个变为 18 个。参数重训练使用 Adam 方法进行优化，由于重训练前的网络参数样本已经处于极值点附近，进行重训练的学习速率需要设置为较小的数值，实验中取$\epsilon = 3\text{e-}4$。得到的结果如下：

表 4.5 网络参数重训练实验结果

方法	细节	存储空间	推断时间	测试错误率
SGLD	24 个模型进行平均	24 倍	24 倍	1.43%±0.07%
SGLD+重训练	重训练后取预测结果最好的模型	**1 倍**	**1 倍**	**1.35%±0.04%**
SGLD+重训练	重训练后再用 18 个模型的预测结果进行平均	18 倍	18 倍	2.13%±0.39%

从表 4.5 中可看出，在 SGLD 方法的基础上进行网络参数重训练，使用最优模型进行预测，又能在 SGLD 方法的基础上进一步带来 6%的测试错误率相对降低，并且存储开销和推断时间都回到了一倍，在各个方面都要强于单纯的 SGLD 方法。然而，在网络参数重训练后再进行模型平均，效果反而不如重训练之前，分析原因在于经过重训练的网络参数样本的分布已不再满足网络参数的后验分布概率，估计预测分布时蒙特卡洛积分的结果产生偏差，导致结果变差。这也说明网络参数重训练方法已经脱离了基于采样的贝叶斯神经网络框架，成为了一种全新的结合 SG-MCMC 采样和随机优化方法的神经网络训练方法。

当网络剪枝和网络参数重训练同时应用时，得到的实验结果如下：

表 4.6 网络剪枝和网络参数重训练结合的实验结果

方法	细节	存储空间	推断时间	测试错误率
SGLD+剪枝+重训练	剪枝比例 94%，之后进行重训练取预测结果最好的模型	0.06 倍	1 倍	1.39%±0.09%
SGD+剪枝+重训练	Han 在论文[29]中给出的结果多次重复剪枝和重训练的过程	0.08 倍	1 倍	1.59%

在网络剪枝和网络参数重训练同时应用时，网络剪枝的剪枝比例可以进一步提高，从 92%提高到 94%而不损失预测准确度。剪枝 94%时，模型规模只有原来的 0.06 倍，而且由于只保留一个最优模型，推断时间也不会额外增加，满足实际应用的需求。第 2 行的结果是 Han 在 15 年的网络压缩研究工作[29]中，和我的实验具有相同的网络结构、训练时长，具备可比性的一组结果。在基于 SGLD 的贝叶斯神经网络框架下应用网络剪枝和网络参数重训练得到的结果，比 Han 使用传统 SGD 方法在训练时多次重复权重剪枝和重训练过程的压缩倍数还要高，并且还在他的测试错误率基础上，得到了 12%的相对降低。

综合以上实验结果，基于网络剪枝的模型压缩和网络参数重训练可以较好地解决贝叶斯神经网络应用时两大难题，为基于 SG-MCMC 采样的贝叶斯神经网络在实际问题中的应用提供了途径。

第5章　结论与展望

本研究中，主要进行了基于 SG-MCMC 采样的贝叶斯神经网络的原理总结和在实际应用中的研究这两方面工作，针对解决贝叶斯神经网络在实际应用中存在模型存储所需空间大、推断时计算时间长的两大难点，取得了以下三点主要成果：

1. 提出贝叶斯神经网络剪枝方法，对神经网络进行基于网络剪枝的模型压缩，存储剪枝压缩后的网络参数样本，用于推断，解决模型的存储问题。实验结果表明，通过剪掉小权重值的网络连接，神经网络可以被压缩到原来大小的 0.06 倍左右而不损失预测准确度，可以在保证精度的同时极大节约存储开销。
2. 提出网络参数重训练方法，解决推断时需要进行模型平均导致计算时间开销过大的问题。经过重训练的单个最优模型在预测中的表现，超过了之前多个模型平均的结果。这种方法脱离了基于采样的贝叶斯神经网络必须做模型平均的限制，利用了 SG-MCMC 采样对于参数空间的有效探寻，为神经网络训练时在参数空间中的寻优提供了一种新思路与新方法。
3. 网络参数重训练也可以和基于网络剪枝的模型压缩方法结合使用，压缩后再进行重训练可以得到更高的压缩比率，并伴随推断时预测准确率上的提升。在基于 SG-MCMC 采样、网络剪枝和网络参数重训练的新框架下，只使用一个规模为原有模型规模 6%大小的压缩模型，就能在 MNIST 手写体识别问题中得到比基准结果相对降低 12%的识别错误率，比原先基于 SG-MCMC 采样的贝叶斯神经网络框架下使用 24 个模型平均的结果还要好 3%。

本研究仍然存在很多可扩展的研究方向。首先，对于模型压缩的研究，只使用了最简单的基于权重阈值进行网络剪枝，以剪掉参数的百分比作为模型压缩比的唯一来源。然而，神经网络模型压缩的空间远不止于此，在 Han 于 2016 年发表的工作[33]中，通过结合剪枝、量化和哈夫曼编码的深度压缩方法，将深度卷积神经网络压缩了将近 50 倍。在贝叶斯神经网络中，如何能利用模型平均带来的鲁棒性，更好地进行模型压缩，值得进行深入挖掘。

针对本研究中提出的基于 SG-MCMC 采样和网络重训练的神经网络训练方法，我已经在 MNIST 手写体识别任务中证明了其有效性。但这种方法能否成为一种普适性的模型训练方法，仍然需要通过将其应用于其他深度学习任务，从多方面多角度考察其性能，才能下定论。另一方面，作为一种新方法，也需要在理

论层面进行完善，为实验结果提供充分的理论解释。通过采样方法和优化方法结合，从而更好地解决模型训练中的优化任务，可以算作是本研究衍生的一个颇具潜力的研究方向。

最后一点，本研究在试图解决基于采样的贝叶斯神经网络的应用难题时，其实没有完全依附于贝叶斯的理论框架下，而是另辟蹊径，从两个模型压缩和重训练这两个相对平行的研究，对贝叶斯神经网络的应用产生了促进作用。如何能对基于采样的贝叶斯神经网络框架本身进行研究，通过纯粹的贝叶斯方式解决贝叶斯神经网络应用中遇到的问题，令人期待。

插图索引

图 1.1 经典神经网络和贝叶斯神经网络的区别 2
图 2.2 SG-MCMC 采样方法和随机优化方法的关系总结 17
图 3.1 3 维参数空间中 L2 正则和 L1 正则等值面示意图 20
图 4.1 层次化编程结构示意图 26
图 4.2 采样实验中的真实概率密度函数 27
图 4.3 采样 10000 个样本时的估计结果 28
图 4.4 采样 3000 个样本时的估计结果 28
图 4.5 HMC 的曲线回归结果 30
图 4.6 SGHMC 的曲线回归结果 30
图 4.7 SGLD 的曲线回归拟合结果 31
图 4.8 pSGLD 的曲线回归拟合结果 31
图 4.9 不同正则形式、正则强度下使用 SGLD 方法的网络剪枝结果 35

表格索引

表 2.1 Metropolis-Hastings 算法流程 13
表 2.2 SGLD 算法流程 15
表 3.1 网络剪枝流程 23
表 3.2 网络参数重训练流程 24
表 4.1 实验环境 25
表 4.2 MNIST 测试错误率对比 33
表 4.3 基于 PTB 数据集的语言模型建模实验结果对比 35
表 4.4 网络剪枝实验结果 36
表 4.5 网络参数重训练实验结果 37
表 4.6 网络剪枝和网络参数重训练结合的实验结果 38
附表 1 pSGLD 算法流程 49
附表 2 SGHMC 算法流程 50
附表 3 SGNHT 算法流程 50
附表 4 SANTA 算法流程 51

参考文献

[1] Blundell C, Cornebise J, Kavukcuoglu K, et al. Weight uncertainty in neural networks[J]. arXiv preprint arXiv:1505.05424, 2015.

[2] Gal Y. Uncertainty in Deep Learning[D]. PhD thesis, University of Cambridge, 2016.

[3] Tishby N, Levin E, Solla S A. Consistent inference of probabilities in layered networks: Predictions and generalization[C]//IJCNN International Joint Conference on Neural Networks. 1989, 2: 403-409.

[4] Buntine W L, Weigend A S. Bayesian back-propagation[J]. Complex systems, 1991, 5(6): 603-643.

[5] Denker J S, Lecun Y. Transforming neural-net output levels to probability distributions[C]//Advances in neural information processing systems. 1991: 853-859.

[6] MacKay D J C. A practical Bayesian framework for backpropagation networks[J]. Neural computation, 1992, 4(3): 448-472.

[7] Neal R M. Bayesian learning via stochastic dynamics[J]. Advances in neural information processing systems, 1993: 475-475.

[8] Neal R M. Bayesian learning for neural networks[M]. Springer Science & Business Media, 2012.

[9] Handbook of markov chain monte carlo[M]. Chapman and Hall/CRC, 2011.

[10] Hinton G E, Van Camp D. Keeping the neural networks simple by minimizing the description length of the weights[C]//Proceedings of the sixth annual conference on Computational learning theory. ACM, 1993: 5-13.

[11] Barber D, Bishop C M. Ensemble learning in Bayesian neural networks[J]. NATO ASI SERIES F COMPUTER AND SYSTEMS SCIENCES, 1998, 168: 215-238.

[12] Graves A. Practical variational inference for neural networks[C]//Advances in Neural Information Processing Systems. 2011: 2348-2356.

[13] Gal Y, Ghahramani Z. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning[C]//International Conference on Machine Learning. 2016: 1050-1059.

[14] Hernández-Lobato J M, Adams R. Probabilistic Backpropagation for Scalable

Learning of Bayesian Neural Networks[C]//ICML. 2015: 1861-1869.

[15] Welling M, Teh Y W. Bayesian learning via stochastic gradient Langevin dynamics[C]//Proceedings of the 28th International Conference on Machine Learning (ICML-11). 2011: 681-688.

[16] Chen T, Fox E, Guestrin C. Stochastic gradient hamiltonian monte carlo[C]//International Conference on Machine Learning. 2014: 1683-1691.

[17] Ding N, Fang Y, Babbush R, et al. Bayesian sampling using stochastic gradient thermostats[C]//Advances in neural information processing systems. 2014: 3203-3211.

[18] Li C, Chen C, Carlson D, et al. Preconditioned stochastic gradient Langevin dynamics for deep neural networks[C]//Thirtieth AAAI Conference on Artificial Intelligence. 2016.

[19] Chen C, Carlson D, Gan Z, et al. Bridging the gap between stochastic gradient MCMC and stochastic optimization[C]//Proc. of AISTATS. 2016.

[20] Li C, Stevens A, Chen C, et al. Learning Weight Uncertainty with Stochastic Gradient MCMC for Shape Classification[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 5666-5675.

[21] Gan Z, Li C, Chen C, et al. Scalable Bayesian Learning of Recurrent Neural Networks for Language Modeling[J]. arXiv preprint arXiv:1611.08034, 2016.

[22] Ahn S. Stochastic Gradient MCMC: Algorithms and Applications[J]. 2015.

[23] Ma Y A, Chen T, Fox E. A complete recipe for stochastic gradient MCMC[C]//Advances in Neural Information Processing Systems. 2015: 2917-2925.

[24] Bridle J S. Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition[M]//Neurocomputing. Springer Berlin Heidelberg, 1990: 227-236.

[25] Robbins H, Monro S. A stochastic approximation method[J]. The annals of mathematical statistics, 1951: 400-407.

[26] Sato I, Nakagawa H. Approximation Analysis of Stochastic Gradient Langevin Dynamics by using Fokker-Planck Equation and Ito Process[C]//Icml. 2014: 982-990.

[27] Scardapane S, Comminiello D, Hussain A, et al. Group Sparse Regularization for Deep Neural Networks[J]. arXiv preprint arXiv:1607.00485, 2016.

[28] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural computation, 1997, 9(8): 1735-1780.

[29] Han S, Pool J, Tran J, et al. Learning both weights and connections for efficient

neural network[C]//Advances in Neural Information Processing Systems. 2015: 1135-1143.

[30] Abadi M, Agarwal A, Barham P, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems[J]. arXiv preprint arXiv:1603.04467, 2016.

[31] LeCun Y. The MNIST database of handwritten digits[J]. http://yann. lecun. com/exdb/mnist/, 1998.

[32] Mikolov T, Karafiát M, Burget L, et al. Recurrent neural network based language model[C]//Interspeech. 2010, 2: 3.

[33] Han S, Mao H, Dally W J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding[J]. arXiv preprint arXiv:1510.00149, 2015.

致 谢

首先特别感谢我的导师欧智坚老师。从我最初的选题，到每一步研究的方向，以及研究过程中遇到的很多问题，欧老师都会事无巨细地与我进行充分讨论，帮助我理清思路，攻克难关。欧老师对于科学研究的激情与热情，以及他多年来坚守的严谨的学术作风，无不对我产生了深远的影响，让我在科研内外都获益良多，真的特别感谢欧老师这一年来为我付出的时间与心血。

其次，感谢我实验室的所有师兄师姐，在我刚刚进入实验室时主动和我交流，帮助我融入这个新环境，得以更好地进行科学研究。

最后，也要感谢我的父母、我的室友叶铭青和同班同学唐紫健，以及无 38 班的秦睿同学，在毕设期间给予了我很多帮助，也在我进行论文写作时为我提出了很多宝贵的建议，谢谢你们。

声 明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签　名：_____　　日　期：_____

附录 A：几种 SG-MCMC 采样算法流程图

附表 1　pSGLD 算法流程

算法 5　pSGLD Algorithm[①]

Input: The value of step size ϵ, precondition λ, grad decay β

Initialize: $v_0 \leftarrow 0$, random θ_0

for $t = 1, 2, \ldots, T$, **do**

Evaluate stochastic gradient:

$$\widetilde{f}_t \leftarrow \frac{N}{n}\sum_{i=1}^{n} \nabla_\theta \log P(y_i|\boldsymbol{x}_i, \theta) + \nabla_\theta \log P(\theta)$$

Evaluate precondition matrix:

$$v_t \leftarrow \beta v_{t-1} + (1-\beta)\widetilde{f}_t^{\,2}$$

$$\mathbf{G}_t^{-1} \leftarrow \mathrm{diag}\left(1/(\lambda \mathbf{I} + \sqrt{v_t})\right)$$

Propose $\eta_t \sim \mathcal{N}(0, \epsilon \mathbf{G}_t^{-1})$

Update parameter:

$$\theta_{t+1} \leftarrow \theta_t + \frac{\epsilon}{2}(\mathbf{G}_t^{-1}\widetilde{f}_t) + \eta_t$$

end for

① 参考文献[18]

附表 2　SGHMC 算法流程

算法 6　SGHMC Algorithm①

Input: step size sequence ϵ_t

Initialize: random $\theta^{(0)}$

for $t = 1, 2, \dots, T$, **do**

Resample momentum $r^{(t)} \sim \mathcal{N}(0, M)$

Assign $(\theta_0, r_0) = (\theta^{(t)}, r^{(t)})$

for $i = 1, \dots, m$, **do**

Evaluate stochastic gradient:

$$\widetilde{f}_t \leftarrow \frac{N}{n}\sum_{i=1}^{n} \nabla_\theta \log P(y_i|\boldsymbol{x}_i, \theta) + \nabla_\theta \log P(\theta)$$

Propose $\eta_t \sim \mathcal{N}\left(0, 2\left(C - \hat{B}\right)\right)\epsilon_t$

Update position and momentum:

$$\theta_i \leftarrow \theta_{i-1} + \epsilon_t M^{-1} r_{i-1}$$

$$r_i \leftarrow r_{i-1} - \epsilon_t \widetilde{f}_t - \epsilon_t C M^{-1} r_{i-1} + \eta_t$$

end for

Accept new state: $(\theta^{(t+1)}, r^{(t+1)}) = (\theta_m, r_m)$ without MH correction

end for

附表 3　SGNHT 算法流程

算法 7　SGNHT Algorithm②

Input: parameters ϵ, A

Initialize: random $\theta^{(0)} \in \mathbb{R}^n, p_0 \sim \mathcal{N}(0, \mathbf{I}), \xi_0 = \mathrm{A}$

for $t = 1, 2, \dots, T$, **do**

Evaluate stochastic gradient $\widetilde{f}_t \leftarrow \frac{N}{n}\sum_{i=1}^{n} \nabla_\theta \log P(y_i|\boldsymbol{x}_i, \theta) + \nabla_\theta \log P(\theta)$

Propose $\eta_t \sim \mathcal{N}(0, \epsilon\mathbf{I})$

Update parameters:

$$p_t \leftarrow p_{t-1} - \xi_{t-1} p_{t-1} \epsilon + \widetilde{f}_t \epsilon + \sqrt{2A}\eta_t$$

$$\theta_t \leftarrow \theta_{t-1} + p_t \epsilon$$

$$\xi_t \leftarrow \xi_{t-1} + (p_t^T p_t - 1)\epsilon$$

end for

① 参考文献[16]
② 参考文献[17]

附表 4　SANTA 算法流程

算法 8　SANTA Algorithm[①]

Input: step size sequence ϵ_t, σ, λ, *burnin*

Initialize: random θ_0, $\mathrm{u}_0 = \mathcal{N}(0, \epsilon_0 \mathbf{I})$, $\alpha_0 = \sqrt{\epsilon_0} C$, $v_0 = 0$

for $t = 1, 2, \ldots, T$, **do**

Evaluate stochastic gradient $\widetilde{f}_t \leftarrow \frac{N}{n} \sum_{i=1}^{n} \nabla_\theta \log P(y_i | \boldsymbol{x}_i, \theta) + \nabla_\theta \log P(\theta)$

$v_0 \leftarrow \sigma v_0 + \frac{1-\sigma}{N^2} \widetilde{f}_t^{\,2}$

$g_t \leftarrow 1 / \sqrt{\lambda + \sqrt{v_t}}$

if $t < burnin$ **then**

Exploration stage:

$\alpha_t \leftarrow \alpha_{t-1} + u_{t-1}^2 - \epsilon_t / \beta_t$

Propose $\eta_t \sim \mathcal{N}(0, \mathbf{I})$

$u_t \leftarrow \frac{\epsilon_t}{\beta_t}(1 - g_{t-1} \cdot g_t) / u_t + \sqrt{\frac{2\epsilon_t}{\beta_t} g_{t-1}} \cdot \eta_t$

else

Refinement stage:

$\alpha_t \leftarrow \alpha_{t-1}$

$u_t \leftarrow 0$

end

$u_t \leftarrow u_t + (1 - \alpha_t) \cdot u_{t-1} - \epsilon_t g_t \cdot \widetilde{f}_t$

$\theta_t \leftarrow \theta_{t-1} + g_t \cdot u_t$

end for

① 参考文献[19]

附录 B：外文资料的调研阅读报告或书面翻译

Literature research of MCMC sampling

Overview of Markov Chain Monte Carlo methods

Markov chain Monte Carlo (MCMC) methods are a class of algorithms for sampling from a probability distribution based on constructing a Markov chain that has the desired distribution of its equilibrium distribution①. The main idea of MCMC sampling method is to construct a Markov chain of which equilibrium distribution is the target distribution (desired posterior distribution). State transition should be guaranteed by detailed balance conditions.

The most classical MCMC sampling method is the Metropolis-Hastings (MH) method. In MH algorithm, before each Markov move we just propose a new state randomly, and decide whether to accept it. Another well-known and widely used classical MCMC sampling method is the Gibbs sampling method. Gibbs sampling can be regarded as a special case of MH where the accept rate is 1 all the time which means we can accept all the state proposed and do the move every time.

These methods are so-called random-walk methods, for the state transition in state space is a highly random behavior. Random walk result in low efficiency, especially in high dimensional space, and the samples generated by classical MCMC methods are usually high correlated.

To deal with the problem, Neal proposed a new MCMC sampling method, the Hybrid Monte Carlo (HMC), which also called the Hamiltonian Monte Carlo method [1]. HMC uses Hamiltonian dynamics to model the state transition process, tries to avoid random walk behavior by introducing an auxiliary momentum vector and implementing Hamiltonian dynamics, so the potential energy function is the target density. By using the gradient information, HMC achieves faster convergence speed and the generation of less correlated samples.

The ability of HMC to do sampling in high dimension meet the need of Bayesian neural network posterior approximation problem. Known as the “golden standard” method in Bayesian posterior sampling, the application of HMC results in some good results in Bayesian neural network [2].

HMC is base of other dynamics-based MCMC sampling algorithms. However, the implementing of HMC need to compute the gradient of energy function over the whole training data set, which can be very computational expensive for a large scale problem.

① Definition of Markov Chain Monte Carlo from Wikipedia.

As the data set become larger and larger, this short come of HMC is getting more and more unbearable. Recently, a series of new dynamics-based MCMC sampling algorithms associated with stochastic approximation (SA) have been studied, known as the stochastic gradient Markov Chain Monte Carlo (SG-MCMC) methods. In SG-MCMC use stochastic approximation methods to calculate stochastic gradient, and based on this stochastic gradient to do transition in the parameter space instead of the true gradient over whole data set.

Stochastic Gradient Langevin Dynamics is the first SG-MCMC sampling method, proposed by Teh and Neal in 2011[3]. SGLD combined the Langevin dynamics and stochastic approximation, can generate samples converge to the true posterior distribution rather than a single mode by just using stochastic gradient. After SGLD various SG-MCMC sampling methods have been proposed, including SGHMC[5], SGNHT[6], pSGLD[7] and SANTA[8] etc., which can all be regarded as extensions of SGLD method. The use of these SG-MCMC method in Bayesian neural network result in the breakthough of SG-MCMC sampling based Bayesian neural network.

Details of Hamiltonian/Hybrid Monte Carlo

HMC is the most developed and widely used dynamics-based sampling method, which is better than random-walk and can be used in high-dimensional space sampling, is known as the base of other dynamics-based MCMC sampling algorithms.

We can know why and how HMC works, ways to do discretization, how to tune parameters, why HMC works well and comparison result (eg. Compare with random-walk, LMC), something about Langevin Monte Carlo (LMC) and the relationship between LMC and HMC according to Neal's work published in 2012[3]. The details are as followed:

1. Why call "hybrid"

On the one hand, HMC alternates Gibbs & Metropolis steps. On the other hand, HMC combines MCMC method with dynamics sampling method. In fact, "Hamiltonian" is a more specific and descriptive name of this method, which is widely used today.

2. Properties of Hamiltonian dynamics

Reversibility: the transition from state t to state $t+s$ can be reversed by negating momentum p, which leads to leave the desired distribution invariant.

Conservation of Hamiltonian: the total amount of potential energy and kinetic energy maintains invariant.

Volume preservation: Hamiltonian dynamics preserves volume in (q,p) space.

Note that the relationship between proposal and transition as following:

$$(q,p) \xrightarrow{\text{proposal a new } p} (q, p_{(0)}^{new}) \xrightarrow{\text{deterministic transition}} (q^{new}, p_{(L)}^{new}) \xrightarrow{\text{MH correction}} (q^*, p^*)$$

The proposal step draws p from $\mathcal{N}(0,1)$, and the acceptance probability is one. How to draw p to sample better should be a problem to be studied.
The transition step is deterministic since the gradient is deterministic for a fixed state. The reversibility means the reversibility of this transition step.

3. The leapfrog method used in HMC

A method to make discretization of continuous-time system in the transition step.
ε: stepsize, determine the error or "accuracy". $\text{Error} \sim O(\varepsilon^2)$.
L: n_step, the number of leapfrog iteration. Set a larger L to better avoid random-walk effect.
εL: the "distance" between two state.

Reference

[1] Neal R M. Probabilistic inference using Markov chain Monte Carlo methods[J]. 1993.

[2] Neal R M. Bayesian learning for neural networks[M]. Springer Science & Business Media, 2012.

[3] Handbook of markov chain monte carlo[M]. Chapman and Hall/CRC, 2011.

[4] Welling M, Teh Y W. Bayesian learning via stochastic gradient Langevin dynamics[C]//Proceedings of the 28th International Conference on Machine Learning (ICML-11). 2011: 681-688.

[5] Chen T, Fox E, Guestrin C. Stochastic gradient hamiltonian monte carlo[C]//International Conference on Machine Learning. 2014: 1683-1691.

[6] Ding N, Fang Y, Babbush R, et al. Bayesian sampling using stochastic gradient thermostats[C]//Advances in neural information processing systems. 2014: 3203-3211.

[7] Li C, Chen C, Carlson D, et al. Preconditioned stochastic gradient Langevin dynamics for deep neural networks[C]//Thirtieth AAAI Conference on Artificial Intelligence. 2016.

[8] Chen C, Carlson D, Gan Z, et al. Bridging the gap between stochastic gradient MCMC and stochastic optimization[C]//Proc. of AISTATS. 2016.